SMART CAMPUS SUSTAINABILITY & CARBON TRACKER

A PROJECT REPORT

Submitted by

KANISHKA S

(240701227)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

October, 2026

BONAFIDE CERTIFICATE

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Certified that this project report titled "Smart Campus Sustainability & Carbon Tracker" is the bonafide work of **KANISHKA S (240701227)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. MANORANJINI J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. BHUVANESWARAN BALASUBRAMANIAN
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

_____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, KANISHKA S (240701227), hereby declare that the project report titled “Smart Campus Sustainability & Carbon Tracker”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report, including its forms, reports, dashboard page, Deluge scripts, workflows and integrations, was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
KANISHKA S
240701227

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. Bhuvaneswaran Balasubramanian, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and its learning resources, and I thank my family and friends for their unwavering support.

KANISHKA S

ABSTRACT

Colleges consume large amounts of electricity and water and generate steady streams of waste, yet the data about them is usually kept in registers and spreadsheets, so management cannot easily see which buildings use the most resources or what carbon emissions result. The Smart Campus Sustainability & Carbon Tracker is a low-code application built on Zoho Creator that brings these records into one system. Zoho Creator was chosen because its forms, reports, page builder, Deluge scripting, integrations and mobile app are available on a single hosted platform, which allowed one student developer to build a complete application within a semester.

The application has eight forms: Buildings, Energy Consumption, Water Consumption, Waste Management, Green Activities, Student Participation, Sustainability Complaints and a Campus Sustainability AI Assistant. Energy, water, waste, activity and complaint records are linked to buildings through lookup fields. Form workflows written in Deluge estimate carbon emissions for each electricity record as kilowatt-hours multiplied by an emission factor, classify records as normal or high consumption, warn the user above 1000 kWh and calculate the recycling percentage of every waste record. A Zia prediction field adds a predicted carbon value, and each new complaint is forwarded to Zoho CRM. Twelve reports (list, kanban, pivot chart and pivot table) and a Sustainability Dashboard with column, bar, pie and line charts, a carbon gauge and key totals summarise the data. A rule-based AI Assistant implemented as Deluge functions answers questions on energy, carbon, water, waste, activities and complaints, and an OpenAI connection is configured for future language-model answers.

The application was reviewed against 24 test cases and the findings are recorded openly, together with proposed fixes. It replaces scattered spreadsheets with validated, linked and summarised records and provides a base for monitoring campus sustainability.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Carbon Footprint; Campus Sustainability; Dashboard

PROJECT AT A GLANCE

Table G.1: Project at a Glance

Attribute	Details
Project Title	Smart Campus Sustainability & Carbon Tracker
Application Domain	Education – campus sustainability, energy, water and waste management
Student Name & Register No.	Kanishka S – 240701227
Zoho Creator Edition	Zoho Creator (India data centre, zoho.in)
Application Name (Link Name)	Smart Campus Sustainability & Carbon Tracker (smart-campus-sustainability-carbon-tracker)
Application URL	https://creatorapp.zoho.in/kanishka.s.2024.cse_rajal703/smart-campus-sustainability-carbon-tracker
Demo Credentials (per role)	Administrator: [to be added] Data Entry Staff: [to be added] Student: [to be added]
Number of Forms / Subforms	8 / 0
Number of Reports (by type)	List: 9 Kanban: 1 Pivot Chart: 1 Pivot Table: 1
Number of Pages / Dashboards	1 (Sustainability Dashboard)
Number of Workflows	Form: 8 Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	5 functions (askCampusAI, getOpenAIAnswer, getSustainabilityInsights, getTotalEnergy, testOpenAIConnection)
Integrations	Zoho CRM (createRecord), OpenAI connector and connection (bearer token), Zia prediction field
Roles & Profiles Defined	Administrator, Write, Read, Developer, Customer profiles; CEO role
Configuration Export	Version 1.0, generated 21 October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 7
2.5 Emerging Trends in Low-Code Development 8
2.6 Proposed System 8
REQUIREMENT ANALYSIS 9
3.1 Stakeholders and User Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 10
3.5 Use Case Model 11
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 18
IMPLEMENTATION 20
5.1 Application Setup 20
5.2 Forms and Data Capture 20
5.3 Reports 21

5.4 Pages and Dashboards 22
5.5 Deluge Scripting 23
5.6 Workflows and Automation 25
5.7 Integrations 26
5.8 User Management and Role-Based Access 27
5.9 AI and Analytics Features 28
5.10 Pro-Code Extension 29
5.11 Mobile and Portal Access 29
TESTING AND DEBUGGING 31
6.1 Testing Strategy 31
6.2 Test Cases 31
6.3 Debugging Deluge Scripts 32
6.4 Defect Log and Review Findings 33
6.5 User Acceptance Testing 34
DEPLOYMENT, SECURITY AND GOVERNANCE 35
7.1 Environments and Deployment Strategy 35
7.2 Performance Optimisation 36
7.3 Security Features 36
7.4 Governance and Administration 37
RESULTS AND DISCUSSION 38
8.1 Application Output 38
8.2 Achievement of Objectives 39
8.3 Comparison with the Existing System 40
8.4 Limitations of the Low-Code Approach 40
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 40
CONCLUSION AND FUTURE ENHANCEMENTS 42
9.1 Conclusion 42
9.2 Future Enhancements 42
9.3 Learning Outcomes and Career Relevance 43
REFERENCES 44
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 45
APPENDIX B – COURSE OUTCOME MAPPING 47
APPENDIX C – COMPLETE DELUGE SOURCE CODE 48
APPENDIX D – USER MANUAL 54
D.1 Accessing the Application 54
D.2 Instructions for Administrators and Data Entry Staff 54
D.3 Instructions for Students and Staff 54
APPENDIX E – EVALUATION RUBRIC 55

APPENDIX F – VIVA-VOCE RECORD SHEET.....57

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application6
Table 2.2: Summary of Literature Survey....7
Table 3.1: Stakeholders and User Roles....9
Table 3.2: Functional Requirements....9
Table 3.3: Non-Functional Requirements....10
Table 3.4: Hardware and Software Requirements....10
Table 3.5: Use Case Description – Record Energy Consumption....11
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – Buildings Form....14
Table 4.3: Field Specification – Energy Consumption Form....14
Table 4.4: Field Specification – Water Consumption Form....15
Table 4.5: Field Specification – Waste Management Form....15
Table 4.6: Field Specification – Green Activities Form....15
Table 4.7: Field Specification – Student Participation Form....16
Table 4.8: Field Specification – Sustainability Complaints Form....16
Table 4.9: Field Specification – Campus Sustainability AI Assistant Form....17
Table 4.10: Role–Permission Matrix (Intended Design)....19
Table 5.1: Reports Implemented....21
Table 5.2: Components of the Sustainability Dashboard....23
Table 5.3: Deluge Scripts Implemented....23
Table 5.4: Workflows Implemented....25
Table 5.5: Integrations Implemented....26
Table 6.1: Test Cases....31
Table 6.2: Defect Log and Review Findings....33
Table 6.3: User Acceptance Feedback (to be completed after the demonstration)....34
Table 7.1: Deployment Checklist....35
Table 7.2: Performance Optimisation Measures....36
Table 7.3: Security Controls Implemented....36
Table 8.1: Mapping of Objectives to Outcomes....39
Table 8.2: Existing System versus Proposed Application....40
Table A.1: Feature Coverage Checklist....45
Table B.1: Mapping of Course Outcomes to Report Chapters....47
Table E.1: Evaluation Rubric for the Individual Project....55
Table E.2: Performance Level Descriptors....55
Table F.1: Viva-Voce Questions and Responses....57
Table F.2: Marks Summary....57

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of Campus Sustainability Data Collection 5
Figure 3.1: Use Case Diagram of the Smart Campus Sustainability & Carbon Tracker 11
Figure 4.1: System Architecture of the Smart Campus Sustainability & Carbon Tracker 12
Figure 4.2: Entity–Relationship Diagram of the Smart Campus Sustainability & Carbon Tracker 13
Figure 4.3: Process Flow for Energy Record Validation 17
Figure 4.4: Navigation Structure of the Application 18
Figure 5.1: Application Home – Navigation Sidebar with Sections Expanded 20
Figure 5.2: Energy Consumption Form in the Form Builder 21
Figure 5.3: Energy Consumption Form in Live Mode – Estimated CO_2 and Status Filled on Submission 21
Figure 5.4: Water Pivot Chart Reports 22
Figure 5.5: Sustainability Dashboard 23
Figure 5.6: Workflow List of the Application 26
Figure 5.7: Connection and Integration Settings 27
Figure 5.8: Profiles and Roles of the Application 28
Figure 5.9: Campus Sustainability AI Assistant – Question and Response 29
Figure 5.10: Application on the Zoho Creator Mobile App 30
Figure 6.1: Running a Function with Execute in the Deluge Editor 33
Figure 7.1: Environments – Publish to Stage and Production 35
Figure 7.2: Application Logs 37
Figure 8.1: Scenario Step 1 – Energy Record Saved with Estimated CO_2 38
Figure 8.2: Scenario Step 2 – Dashboard Updated 39
Figure 8.3: Scenario Step 3 – AI Assistant Response 39

LIST OF ABBREVIATIONS

Table X.1: Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CO	Course Outcome
CO_2	Carbon Dioxide
CRM	Customer Relationship Management
DS	Deluge Script
FR	Functional Requirement
GHG	Greenhouse Gas
INR	Indian Rupee
IoT	Internet of Things
kWh	Kilowatt-hour
KPI	Key Performance Indicator
LCDP	Low-Code Development Platform
NFR	Non-Functional Requirement
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Universities and engineering colleges are small towns. A single campus runs hostels, laboratories, libraries, canteens, sports facilities and academic blocks, and together these consume large quantities of electricity and water and produce steady streams of solid, food and electronic waste. Because the electricity supplied to most campuses in India comes largely from a grid that still depends on fossil fuels, every unit of electricity consumed is also linked to greenhouse-gas emissions. Campus sustainability programmes therefore begin with measurement: how much energy and water does each building use, how much waste does it generate and how much of that is recycled, and what are students and staff doing to improve the situation?

At Rajalakshmi Engineering College, where this project was carried out, such information is generated by several different people (facility staff who read meters, housekeeping and canteen teams who handle waste, and student clubs that organise plantation and clean-up drives). In practice it is held in registers, spreadsheets and messaging groups. A complaint about a leaking tap or a light left on overnight travels through informal channels and is rarely tracked to closure.

This project replaces that scattered record-keeping with a single Zoho Creator application, the **Smart Campus Sustainability & Carbon Tracker**. The application stores campus buildings, electricity, water and waste records, green activities, student participation and sustainability complaints in one place, estimates the carbon footprint of every electricity record automatically, summarises everything on a dashboard and answers sustainability questions through a built-in assistant.

1.2 Problem Statement

Campus sustainability data is collected in disconnected registers and spreadsheets, so it is difficult to see how much electricity and water each building uses, how much waste is generated and recycled, what carbon emissions result, which green activities are taking place and which complaints remain unresolved. A system is needed that lets authorised users enter these records once, validates and enriches them automatically, and presents them through reports and a dashboard that decision makers can read without preparing manual summaries.

- **Resource visibility:** electricity and water records are difficult to review when the information is spread across separate files and registers.
- **No carbon view:** electricity use is recorded in kilowatt-hours, but nobody converts it into emissions, so the environmental impact of each building stays invisible.
- **Waste tracking:** waste quantities, disposal methods and recycling performance are not recorded in a consistent structure.
- **Green activities and participation:** plantation drives, clean-ups and awareness programmes are not linked to the students who take part, so contribution cannot be recognised.
- **Complaints:** sustainability concerns such as leakage, overflowing bins or broken lights have no register with priority, owner and resolution tracking.
- **Slow insight:** answering a simple question such as "which record has the highest consumption?" requires opening and filtering several files by hand.

1.3 Objectives

- To design a connected data model and forms for buildings, energy, water, waste, green activities, student participation, sustainability complaints and assistant queries in Zoho Creator.
- To estimate carbon emissions automatically from each electricity record using a configurable emission factor, and to flag high-consumption records.
- To calculate the recycling percentage of every waste record automatically.
- To provide list, kanban and pivot reports and a Sustainability Dashboard that summarise energy, water, waste, trees planted and participation.
- To provide a Campus Sustainability AI Assistant that answers questions about energy, carbon, water, waste, green activities, participation and complaints from the stored data.
- To forward sustainability complaints to Zoho CRM and to predict carbon estimates with a Zia prediction field, demonstrating integration and AI features of the platform.

1.4 Scope of the Project

In scope. The application covers eight forms (Buildings, Energy Consumption, Water Consumption, Waste Management, Green Activities, Student Participation, Sustainability

Complaints and Campus Sustainability AI Assistant), twelve reports, one dashboard page, eight form workflows, five Deluge functions, a Zoho CRM integration for complaints, an OpenAI connector configuration and a Zia prediction field for estimated carbon. It is used through the Zoho Creator web interface and the Zoho Creator mobile application.

Out of scope. The application does not read data directly from smart meters or IoT sensors; all readings are entered by users. It does not generate official sustainability certificates, perform Scope 3 emission accounting, schedule notifications or provide Blueprint or approval processes in this version. The carbon estimate uses one configured emission factor and is intended for awareness and trend monitoring, not for statutory reporting. These items are listed as future enhancements in Chapter 9.

1.5 Justification for a Low-Code Approach

A campus sustainability tracker is a typical internal business application: a handful of related entities, simple calculations, reports and a dashboard. Zoho Creator provides all the building blocks required (forms with lookups, validation workflows, reports, a page builder, the Deluge scripting language, integrations and a mobile client) on a hosted platform [1]. Hosting, authentication, the database and the mobile application did not have to be built, so the effort could be spent on the logic that matters for sustainability: the carbon estimate, the recycling calculation, the high-consumption rule and the assistant.

Low-code platforms aim to let developers and domain experts build applications through visual modelling with a limited amount of hand-written code [2], [3]. That suited this project, which was built by a single student within one semester. Where the visual tools were not enough, targeted code was added: Deluge functions for analytics, form workflows for validation, a REST call through an authorised connection for the OpenAI connector test and the Zoho CRM API for complaint forwarding [4], [5]. The application can therefore be maintained through configuration for routine changes while still supporting custom behaviour.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the

report with future enhancements. The appendices contain the feature coverage checklist, course outcome mapping, the complete Deluge source code, a user manual and the evaluation sheets.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process for collecting campus sustainability information, as followed in most institutions, works as follows:

1. Facility staff note electricity meter readings and water usage in a register or spreadsheet at irregular intervals.
2. Housekeeping and canteen staff record the quantity of waste collected, often without separating recyclable from non-recyclable material.
3. Student clubs and departments organise plantation drives, clean-ups and awareness events and report them through letters or messages.
4. Complaints about leakage, overflowing bins or unnecessary power use are passed on verbally or through messaging groups.
5. At the end of a term, an administrator compiles the scattered figures manually into a report for management.
6. Carbon emissions are rarely calculated at all, because converting each reading needs a separate calculation.

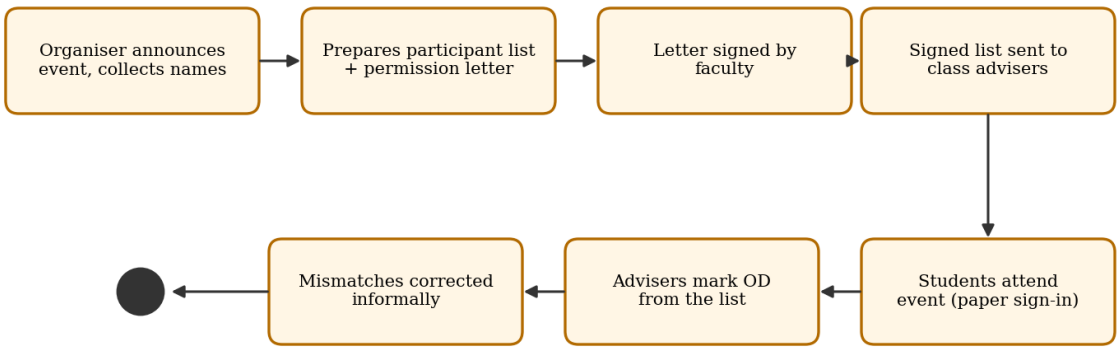


Figure 2.1: As-Is Process Flow of Campus Sustainability Data Collection

2.2 Limitations of the Existing System

- Manual and repeated data entry: the same figures are written in registers, then re-typed into spreadsheets and reports.

- No consolidated view across electricity, water, waste, activities and complaints.
- Carbon impact is not calculated, so high-emitting buildings are not identified.
- No structured complaint tracking: no priority, assignee, status or resolution date.
- Student contribution to green activities is not recorded, so recognition and motivation are lost.
- Paper and spreadsheet records are hard to search, summarise or audit at the end of the year.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Several weeks to a few months (estimate) for a database, back-end API, web front-end, charts and authentication	Built within the semester by one student using forms, reports, a dashboard page and Deluge
Skills required	Full-stack development: back-end framework, database design, front-end charting library, DevOps	Zoho Creator configuration, Deluge scripting and basic REST/CRM API concepts
Hosting & infrastructure	Servers or cloud services must be provisioned, secured and maintained	Hosted by Zoho; no server administration
Dashboards & charts	A charting library must be integrated and data endpoints written	Drag-and-drop page builder with column, bar, pie, line charts, gauge and KPI panels bound directly to forms
Mobile application	A separate Android/iOS app must be built and published	The Zoho Creator mobile app runs the same application
Security & access control	Authentication, roles and data protection implemented by hand	Built-in user management, profiles, roles and personal-data flags
AI features	Machine-learning pipeline and model hosting required	Zia prediction field configured from form data
Customisation limits	Unlimited	Limited to platform components; very custom UI or logic needs scripting
Cost	Hosting and developer time	Per-user subscription after the trial; no separate hosting cost

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, 2020 [7]	Supporting the understanding and comparison of low-code development platforms	Technical survey and feature-based comparison of low-code platforms covering data modelling, UI, logic, integration and deployment.	Gave the vocabulary used to map this project's features and position Zoho Creator among low-code platforms.
2	A. C. Bock and U. Frank, 2021 [2]	Low-Code Platform (Business & Information Systems Engineering)	Characterises low-code platforms, citizen developers and their effect on productivity and agility.	Supports the justification that a single student can deliver an institutional application on a low-code platform.
3	D. Di Ruscio et al., 2022 [3]	Low-code development and model-driven engineering: Two sides of the same coin?	Compares low-code with model-driven engineering, noting shared ideas and differing strengths.	Frames the forms, lookups and workflows used here as models from which the platform generates a running application.
4	M. A. A. Alamin et al., 2021 [8]	An Empirical Study of Developer Discussions on Low-Code Software Development Challenges	Analyses developer questions on low-code platforms; customisation and documentation gaps are recurring themes.	Matches challenges met here, such as limits of the page builder and inconsistent aggregate behaviour in scripts.
5	World Resources Institute and WBCSD, 2004 [9]	The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard	Defines emissions as activity data multiplied by an emission factor and separates direct and purchased-electricity emissions.	Basis of the carbon estimate: kWh consumed multiplied by an emission factor.
6	Central Electricity Authority, Government of India [10]	CO_2 Baseline Database for the Indian Power Sector	Publishes grid emission factors for India that are revised periodically.	Source type for a verified emission factor; the factor configured in this version is a placeholder to be replaced.
7	Universitas Indonesia, UI GreenMetric [11]	UI GreenMetric World University Rankings	Ranks campuses on categories such as energy and climate change, waste, water and education.	Confirms energy, waste, water and awareness as the standard dimensions of campus sustainability, as modelled here.
8	Zoho Corporation [1], [4], [5]	Zoho Creator Help, Deluge Reference and Zoho CRM API documentation	Official documentation of forms, workflows, reports, pages, Deluge and CRM record creation.	Primary technical reference for every component in Chapter 5.

2.5 Emerging Trends in Low-Code Development

Three trends shaped this project. The first is AI-assisted building and AI fields. Zoho Creator offers prediction fields trained on form data, and in this project the Estimated CO_2 Prediction field uses such a model. The second is citizen development combined with pro-code extension. Platforms let non-specialists build most of an application visually while developers add scripts where needed [2], [3]; here, forms, reports and the dashboard are configured visually, while the analytics assistant and the CRM and connector calls are written in Deluge. The third is composable applications that connect to other services; the complaint form forwards each complaint to Zoho CRM, and a connection to an external AI service is configured for future use.

2.6 Proposed System

The Smart Campus Sustainability & Carbon Tracker addresses each limitation of the existing system. Manual spreadsheets are replaced by structured forms with mandatory fields and lookups to a common Buildings master. Carbon impact is estimated automatically whenever an electricity record is saved, and the recycling percentage is computed for every waste record. A single dashboard shows energy by building, water by building, waste by type, the energy trend, participation by activity, average estimated carbon against a gauge, and totals for energy, water, waste and trees planted. Complaints are captured with category, priority, assignee, status and resolution, and are forwarded to Zoho CRM. Finally, the AI Assistant answers plain-language questions about the stored data and gives recommendations for saving electricity and water, reducing waste and organising green activities.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	The application owner and sustainability officer with full access	Maintains the application, manages users and profiles, reviews the dashboard and all reports, sets thresholds and emission factor
Facility / Data Entry Staff	Staff who read meters and record waste, water and activity data (Write profile)	Adds and edits building, energy, water, waste and activity records; resolves complaints
Student / Staff	Campus members who participate in green activities and report concerns	Submits sustainability complaints, takes part in green activities (recorded as participation) and asks the AI Assistant questions
External system: Zoho CRM	Customer-relationship system of the institution	Receives a contact record for each sustainability complaint

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall maintain a master list of buildings with block, department, floors, capacity and status.	Administrator	High
FR-02	The system shall record electricity consumption per building and date with meter reading, units in kWh and cost in INR.	Data Entry Staff	High
FR-03	The system shall estimate carbon emissions for every electricity record as units consumed multiplied by the configured emission factor.	System	High
FR-04	The system shall classify an electricity record as Normal or High Consumption and warn the user when consumption exceeds 1000 kWh.	System	Medium
FR-05	The system shall record water consumption per building with litres used, source and cost.	Data Entry Staff	High
FR-06	The system shall record waste per building with type, total and recycled quantity and disposal method, and calculate the recycling percentage.	Data Entry Staff, System	High
FR-07	The system shall record green activities with type, date, location, participants, trees planted and waste collected.	Data Entry Staff	Medium
FR-08	The system shall record student participation in a green activity with type of participation and green points.	Data Entry Staff	Medium
FR-09	The system shall register sustainability complaints with category, priority, assignee, status and resolution, and forward each new complaint to Zoho CRM.	Student / Staff, System	High
FR-10	The system shall provide list, kanban, pivot chart and pivot table reports of the recorded data.	All roles	High

FR ID	Requirement	Role(s)	Priority
FR-11	The system shall present a dashboard with charts, a gauge and key totals.	Administrator	High
FR-12	The system shall answer sustainability questions typed into the AI Assistant form and store the question, answer and time.	All roles	Medium
FR-13	The system shall predict an estimated carbon value for an electricity record using a prediction model.	System	Low
FR-14	The system shall provide summary insights (total energy, total carbon, average energy and number of high-usage records) through a reusable function.	System	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Calculations for carbon, status and recycling shall be completed within the same form submission without manual steps.
NFR-02	Security	Access shall be limited to authenticated users through Zoho Creator profiles; the e-mail address of a complainant shall be flagged as personal data; external API access shall use an authorised connection.
NFR-03	Usability	The application shall be usable on the Zoho Creator web interface and the mobile app, with picklists and lookups instead of free text wherever possible.
NFR-04	Availability	The application shall be hosted on Zoho's cloud and accessible wherever an internet connection is available.
NFR-05	Maintainability	Shared logic shall be implemented as reusable Deluge functions; field link names shall follow a consistent pattern.
NFR-06	Auditability	Every record shall retain the user and time of creation, and workflow executions shall be traceable through application logs.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (India data centre, creatorapp.zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
Other Zoho services	Zoho CRM (complaint contacts) and Zia prediction (AI field)
External service	OpenAI API connection (bearer-token connector) configured for future language-model answers
Mobile	Zoho Creator mobile app (Android / iOS)
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

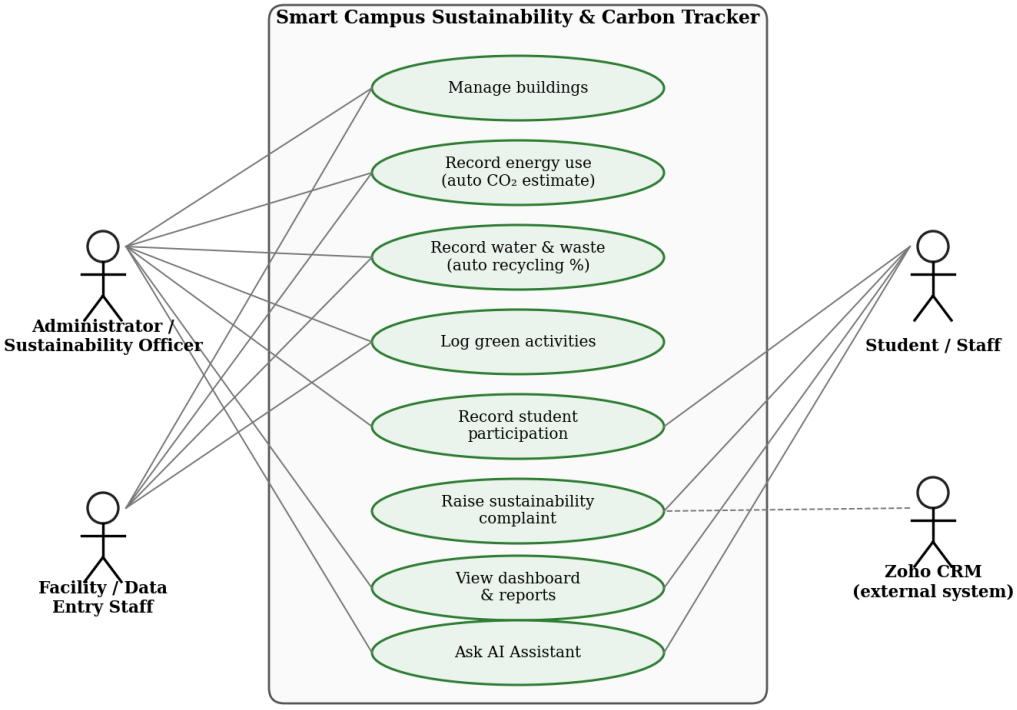


Figure 3.1: Use Case Diagram of the Smart Campus Sustainability & Carbon Tracker

Table 3.5: Use Case Description – Record Energy Consumption

Item	Description
Use Case ID	UC-01
Use Case Name	Record energy consumption and estimate carbon
Actor(s)	Data Entry Staff (primary), System
Pre-condition	The building exists in the Buildings form and the user has the Write profile.
Main Flow	1. The user opens Energy Consumptions > Energy Consumption and enters the record ID, building, date, meter reading, units consumed (kWh) and cost. 2. On submission the validate workflows run. 3. The system calculates Estimated CO_2 = units × emission factor. 4. The system sets Energy Status to Normal when units are 1000 kWh or less. 5. The record is saved and appears in reports and on the dashboard.
Alternate Flow	3a. Units are zero or empty: Estimated CO_2 is set to 0. 4a. Units exceed 1000 kWh: an alert is shown and the submission is cancelled so that the entry can be reviewed.
Post-condition	An energy record with its carbon estimate exists and is reflected in charts and totals.
Related FR	FR-02, FR-03, FR-04, FR-11

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

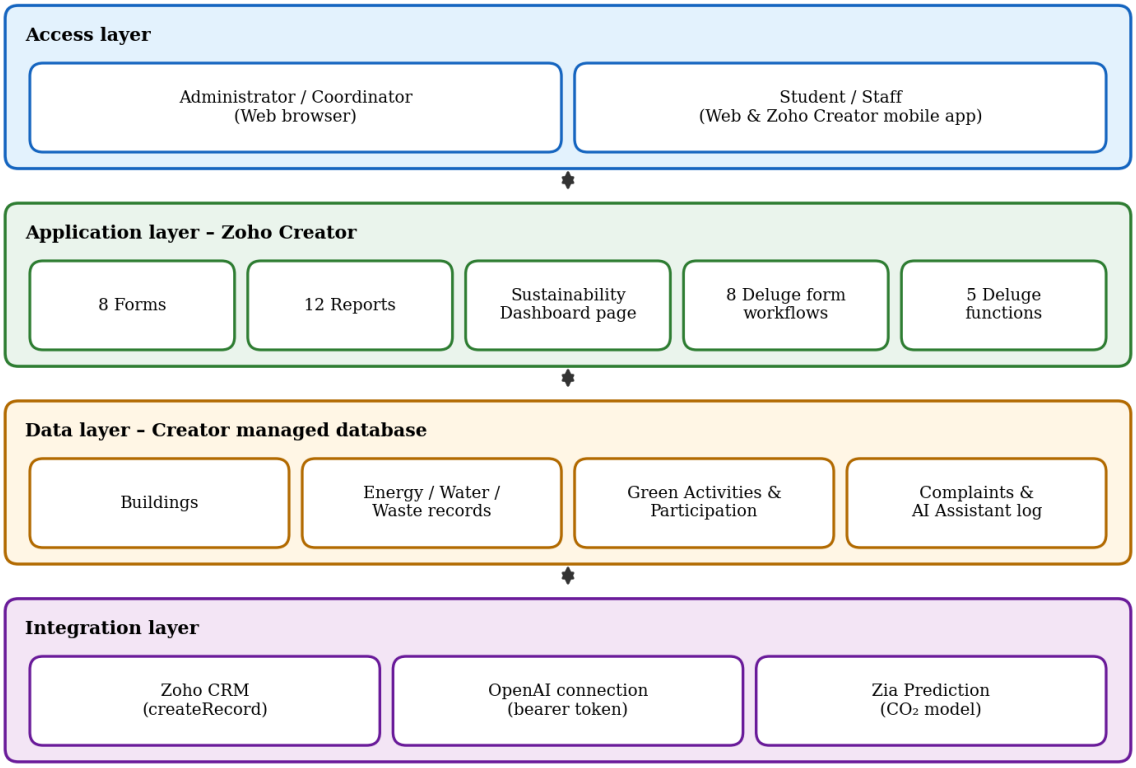


Figure 4.1: System Architecture of the Smart Campus Sustainability & Carbon Tracker

The application is organised in four layers. The access layer consists of the users and their channels: administrators and data-entry staff work on the Zoho Creator web interface, and students and staff may also use the Zoho Creator mobile app. The application layer is the Zoho Creator application itself: eight forms, twelve reports, one dashboard page, eight form workflows and five Deluge functions. Profiles and roles at this layer decide what each user can see.

The data layer is Zoho Creator's managed database, in which the forms are linked by lookup fields to the Buildings form and, for participation, to the Green Activities form. The integration layer connects the application to external services: Zoho CRM through the built-in CRM integration task (createRecord), an OpenAI connector with a bearer-token connection (openai_sustainability_connection) that is tested by a function, and the Zia prediction engine that powers the Estimated CO_2 Prediction field.

4.2 Data Model

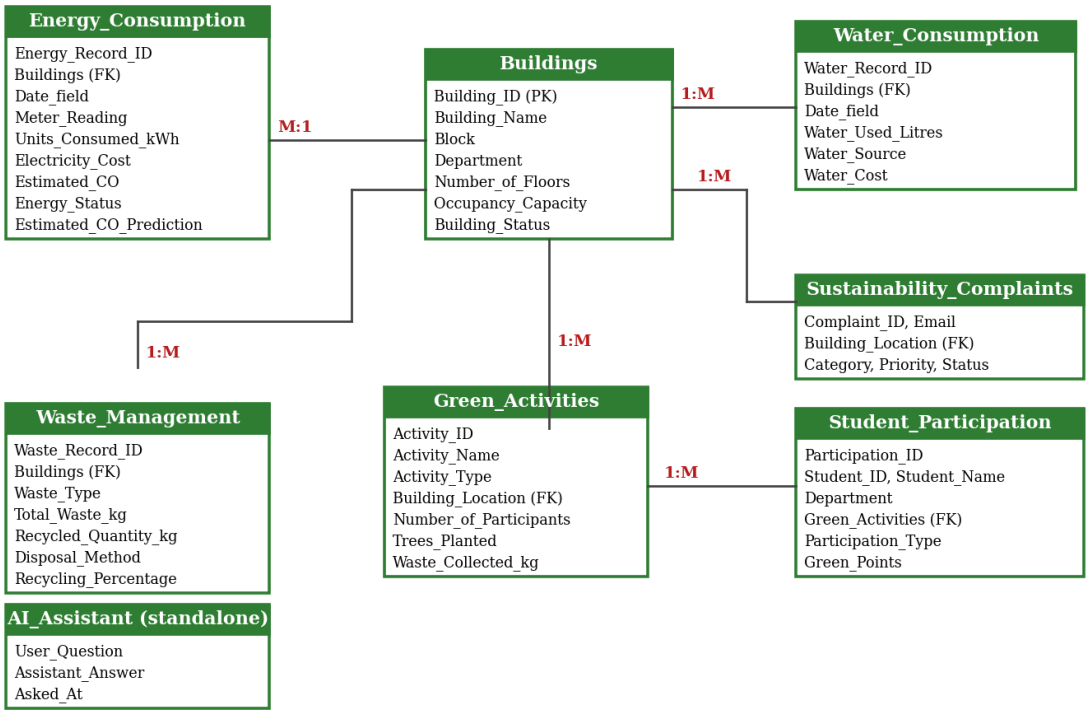


Figure 4.2: Entity–Relationship Diagram of the Smart Campus Sustainability & Carbon Tracker

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Buildings	Building_ID, Building_Name, Block, Department, Building_Status	Energy_Consumption	One-to-Many	Lookup (Buildings) in Energy_Consumption
Buildings	as above	Water_Consumption	One-to-Many	Lookup (Buildings) in Water_Consumption
Buildings	as above	Waste_Management	One-to-Many	Lookup (Buildings) in Waste_Management
Buildings	as above	Green_Activities	One-to-Many	Lookup (Building_Location)
Buildings	as above	Sustainability_Complaints	One-to-Many	Lookup (Building_Location)
Green_Activities	Activity_ID, Activity_Name, Activity_Type, Number_of_Participants	Student_Participation	One-to-Many	Lookup (Green_Activities) in Student_Participation

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Campus_Sustainability_AI_Assistant	User_Question, Assistant_Answer, Asked_At	—	Standalone	Reads all other forms through the askCampusAI function

The Buildings form is the centre of the model. Every energy, water, waste, activity and complaint record refers to a building through a lookup field, so the dashboard can group figures by building name. Each participation record refers to exactly one green activity. The AI Assistant form is deliberately standalone: it stores questions and answers, while the function behind it reads the other forms directly.

4.3 Form Design

Table 4.2: Field Specification – Buildings Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Building ID	Building_ID	Single Line	Yes	Maximum 10 characters
Building Name	Building_Name	Single Line	Yes	Shown in all lookups
Block	Block	Drop Down	Yes	Academic, Administrative, Hostel, Library, Laboratory, Sports, Canteen, Other; default Hostel
Department	Department	Single Line	Yes	Maximum 50 characters
Number of Floors	Number_of_Floors	Number	No	—
Occupancy Capacity	Occupancy_Capacity	Number	No	—
Building Status	Building_Status	Drop Down	Yes	Active, Under Maintenance, Inactive

Table 4.3: Field Specification – Energy Consumption Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Energy Record ID	Energy_Record_ID	Single Line	Yes	—
Buildings	Buildings	Lookup	No	Lookup to Buildings, shows Building Name, ascending
Date	Date_field	Date	Yes	—
Meter Reading	Meter_Reading	Number	No	—
Units Consumed (kWh)	Units_Consumed_kWh	Number	No	Drives CO_2 and status
Electricity Cost	Electricity_Cost	Currency (INR)	No	Indian comma format
Estimated CO_2	Estimated_CO	Decimal	—	Set by script: units × 0.82
Energy Status	Energy_Status	Drop Down	—	Normal / High Consumption; set by script

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Estimated CO_2 Prediction	Estimated_CO_Prediction	Decimal (AI field)	—	Zia prediction; model Energy_Consumption_Model; supporting fields: cost, meter reading, units

Table 4.4: Field Specification – Water Consumption Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Water Record ID	Water_Record_ID	Single Line	Yes	Maximum 50 characters
Buildings	Buildings	Lookup	No	Lookup to Buildings
Date	Date_field	Date	Yes	—
Water Used (Litres)	Water_Used_Litres	Number	Yes	—
Water Source	Water_Source	Drop Down	No	Municipal, Borewell, Tanker, Rainwater, Other
Water Cost	Water_Cost	Currency (INR)	Yes	Indian comma format

Table 4.5: Field Specification – Waste Management Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Waste Record ID	Waste_Record_ID	Single Line	Yes	Maximum 10 characters
Buildings	Buildings	Lookup	No	Lookup to Buildings
Date	Date_field	Date	Yes	—
Waste Type	Waste_Type	Drop Down	No	Food Waste, Plastic, Paper, E-Waste, Metal, Glass, Other
Total Waste (kg)	Total_Waste_kg	Decimal	Yes	—
Recycled Quantity (kg)	Recycled_Quantity_kg	Decimal	Yes	—
Disposal Method	Disposal_Method	Drop Down	Yes	Recycled, Composted, Landfill, Incinerated, Other
Recycling Percentage	Recycling_Percentage	Decimal	—	Set by script: recycled ÷ total × 100

Table 4.6: Field Specification – Green Activities Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Activity ID	Activity_ID	Single Line	Yes	—
Activity Name	Activity_Name	Single Line	Yes	—
Activity Type	Activity_Type	Drop Down	Yes	Tree Plantation, Campus Cleaning, E-Waste Collection, Plastic-Free Campaign, Awareness Program, Energy Conservation, Water Conservation, Other
Date	Date_field	Date	No	—
Building/Location	Building_Location	Lookup	No	Lookup to Buildings

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Department	Department	Single Line	Yes	—
Number of Participants	Number_of_Participants	Number	Yes	—
Trees Planted	Trees_Planted	Number	No	Summed on the dashboard
Waste Collected (kg)	Waste_Collected_kg	Decimal	Yes	—
Description	Description	Multi Line	No	—

Table 4.7: Field Specification – Student Participation Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Participation ID	Participation_ID	Single Line	Yes	Maximum 10 characters
Student ID	Student_ID	Single Line	Yes	Maximum 10 characters
Student Name	Student_Name	Single Line	Yes	Maximum 100 characters
Department	Department	Drop Down	Yes	CSE, IT, ECE, EEE, Mechanical, Civil, Other
Green Activities	Green_Activities	Lookup	Yes	Lookup to Green Activities, shows Activity Name
Date	dATE_field1	Date	No	—
Participation Type	Participation_Type	Drop Down	Yes	Volunteer, Participant, Organizer, Coordinator
Green Points	Green_Points	Number	Yes	Recognition points for the student

Table 4.8: Field Specification – Sustainability Complaints Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Complaint ID	Complaint_ID	Single Line	Yes	—
Reported By	Reported_By	Single Line	Yes	Becomes Last Name of the CRM contact
Email	Email	Email	Yes	Maximum 80 characters; marked as personal data
Building/Location	Building_Location	Lookup	No	Lookup to Buildings
Complaint Category	Complaint_Category	Drop Down	Yes	Water Leakage, Excess Electricity, Waste Overflow, Plastic Waste, Broken Lights, E-Waste, Other
Description	Description	Multi Line	No	—
Date Reported	Date_Reported	Date	No	—
Priority	Priority	Drop Down	No	High, Medium, Low
Assigned To	Assigned_To	Single Line	No	—
Status	Status	Drop Down	Yes	New, Assigned, In Progress, Resolved, Closed
Resolution	Resolution	Multi Line	No	—

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Resolution Date	Resolution_Date	Date	No	—

Table 4.9: Field Specification – Campus Sustainability AI Assistant Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Your Question	User_Question	Multi Line	No	Free-text question
Assistant Answer	Assistant_Answer	Multi Line	No	Filled by the askCampusAI function
Asked At	Asked_At	Date-Time	No	Set to the current time on submission

4.4 Process and Workflow Design

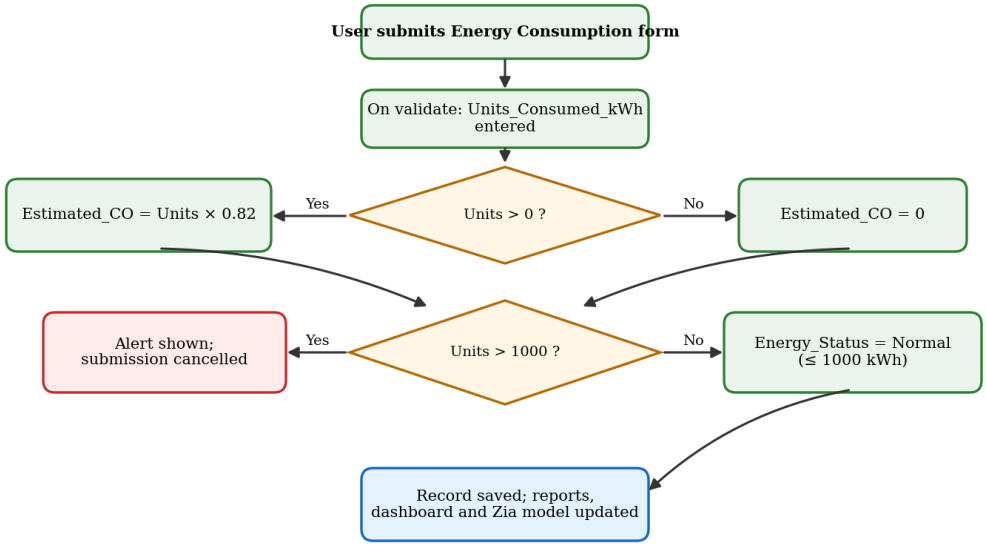


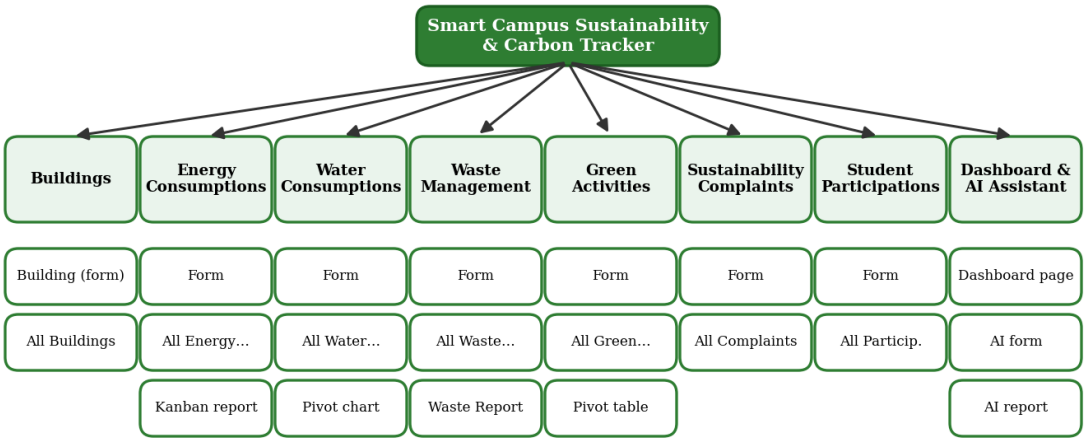
Figure 4.3: Process Flow for Energy Record Validation

The central automated process is the validation of an energy record. When a user submits the Energy Consumption form, four validate workflows run in sequence: the first computes Estimated CO_2 as units multiplied by 0.82 (or zero when no units are entered); the second alerts the user and cancels submission when units exceed 1000 kWh; the third sets Energy Status to High Consumption above 1000 kWh and to Normal otherwise. Waste records follow the same pattern: the Calculate Recycling Percentage workflow divides recycled quantity by total quantity and multiplies by 100, or stores zero when the total is zero.

Two other processes complete the design. When a complaint is saved, an on-success workflow creates a contact in Zoho CRM carrying the complaint details. When the AI Assistant form is

submitted, validate workflows call the askCampusAI function with the question, store the answer and the time, and after a successful save the user is directed to the AI Assistant report.

4.5 User Interface and Navigation Design



Also: App Preferences (Language, Integrations) and Approvals (Pending / Completed Tasks) system sections.

Figure 4.4: Navigation Structure of the Application

The navigation menu has one section for each subject area: Buildings, Energy Consumptions, Water Consumptions, Waste Management, Green Activities, Sustainability Complaints, Student Participations, Sustainability Dashboard and Campus Sustainability AI Assistant. Each section holds the data-entry form and its reports, so a user finds “add” and “view” for a subject side by side. System sections for App Preferences (language selection and integrations) and Approvals (pending and completed tasks) are also present. Zoho Creator hides components for which the user has no permission, so the same menu structure presents a smaller menu to restricted users.

4.6 Access Control Design

Access is designed around three groups of users mapped to Zoho Creator’s built-in profiles. Administrators use the Administrator profile; data-entry staff use the Write profile; and students and staff who only read information use the Read or Customer profile. A role hierarchy with a single CEO role is present, so a user in that role can access the data of all other users. Table 4.10 summarises the intended permissions (A = add, E = edit, V = view, D = delete).

Table 4.10: Role–Permission Matrix (Intended Design)

Component	Administrator	Data Entry Staff (Write)	Student / Staff (Read / Customer)
Form: Buildings	A E V D	A E V	V
Forms: Energy, Water, Waste	A E V D	A E V	V
Forms: Green Activities, Student Participation	A E V D	A E V	A V
Form: Sustainability Complaints	A E V D	A E V	A V
Form: AI Assistant	A E V D	A V	A V
Reports (all)	V	V	V
Page: Sustainability Dashboard	V	V	V

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application “Smart Campus Sustainability & Carbon Tracker” was created on Zoho Creator in the India data centre (creator.zoho.in) under the account kanishka.s.2024.cse_rajal703, with the link name smart-campus-sustainability-carbon-tracker. The application uses the date format dd-MMM-yyyy, the 24-hour time format and the Asia/Kolkata time zone, and its icon is the text “SCS”. Components are grouped into navigation sections (Figure 5.1). A configuration export (.ds file, version 1.0, generated on 21 October 2026) was taken and is the source for the figures and code quoted in this report.

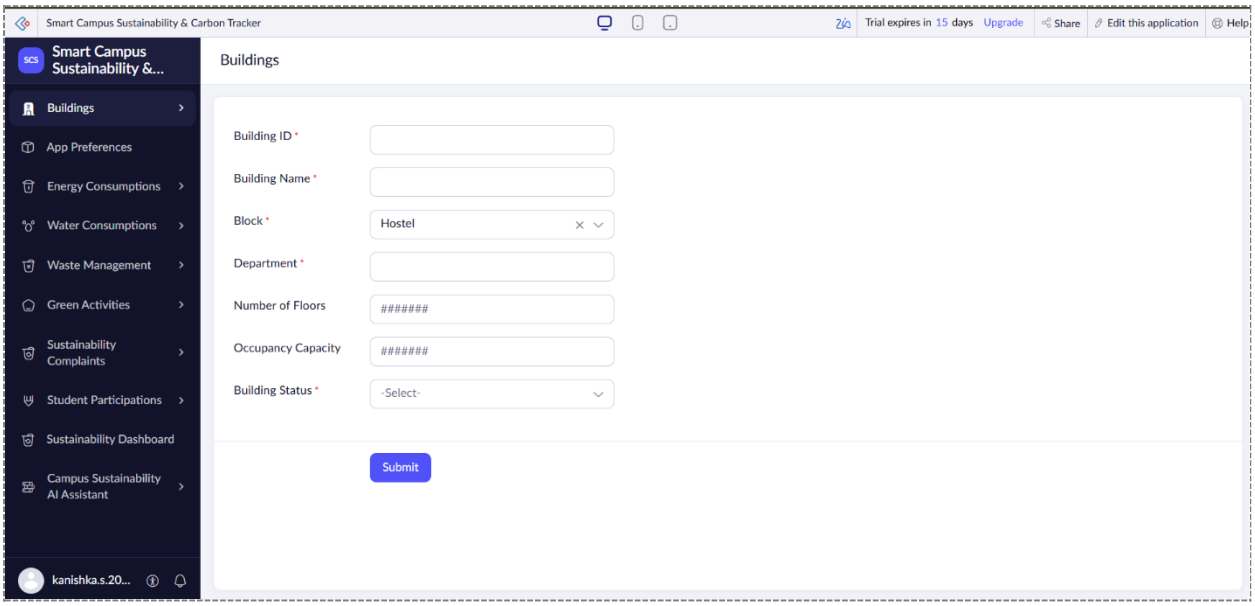


Figure 5.1: Application Home – Navigation Sidebar with Sections Expanded

5.2 Forms and Data Capture

Eight forms were built with the Zoho Creator form builder (Tables 4.2 to 4.9). They use text, number, decimal, date, date-time, e-mail, multi-line, drop-down, lookup and currency fields. Required fields are marked as mandatory, drop-downs restrict values to controlled lists (for example Block, Waste Type, Complaint Category and Priority), and lookups to the Buildings form guarantee that every record refers to an existing building. Cost fields use the Indian rupee currency type with Indian digit grouping. The complainant’s e-mail address is marked as personal data. Each form shows a friendly success message after submission, such as “Energy Consumption Added Successfully”.

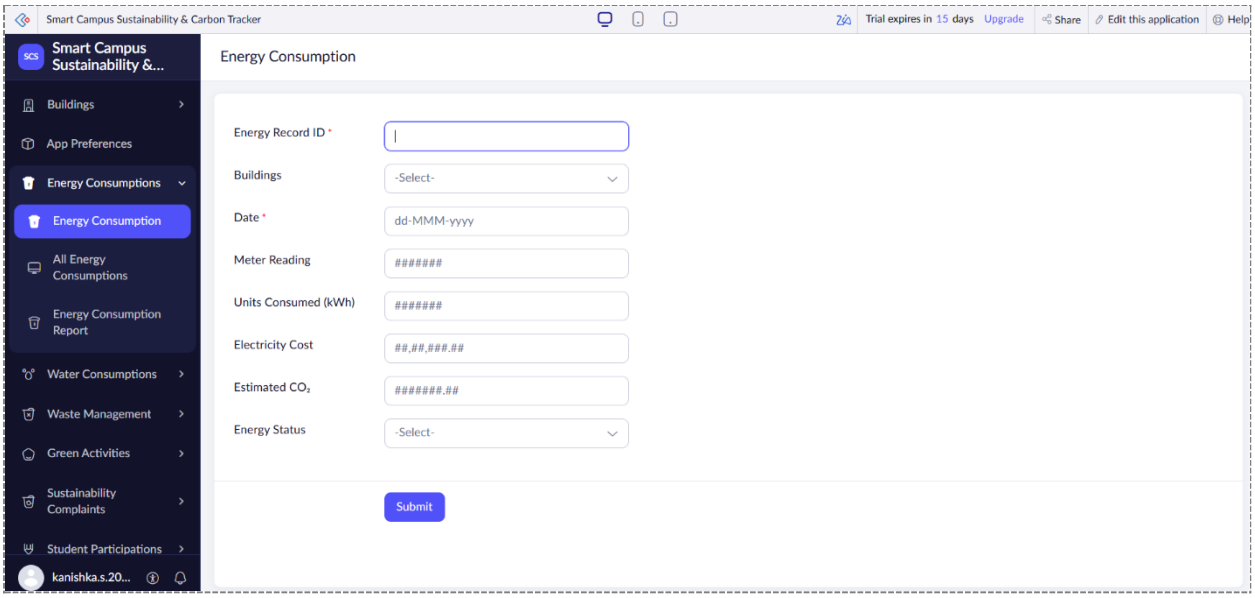


Figure 5.2: Energy Consumption Form in the Form Builder

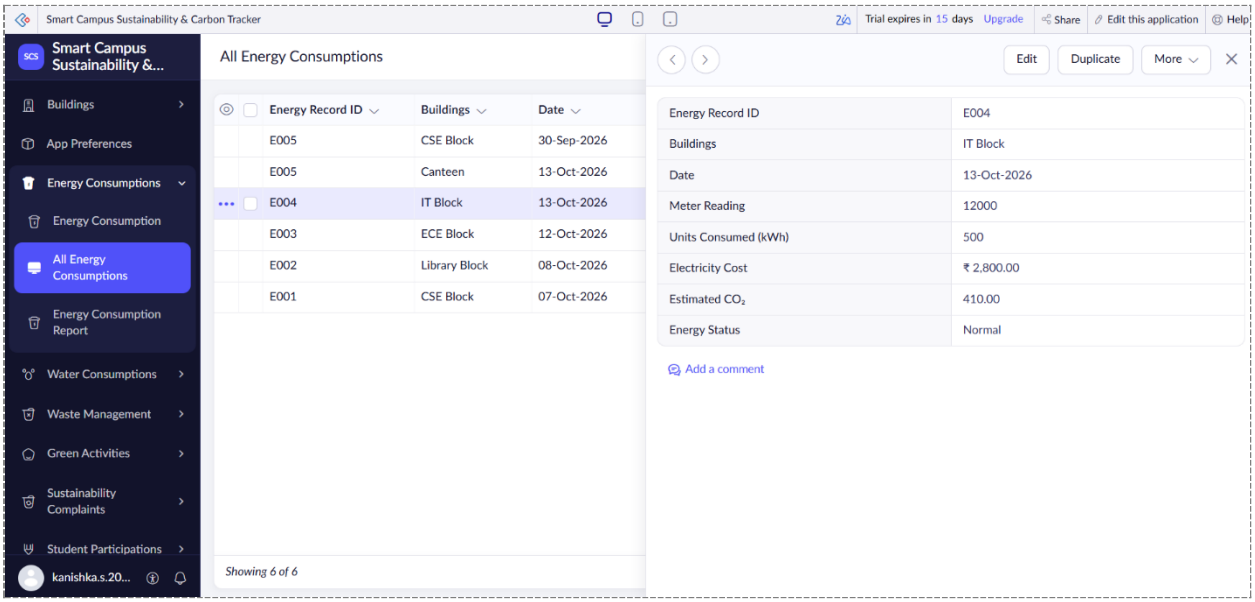


Figure 5.3: Energy Consumption Form in Live Mode – Estimated CO_2 and Status Filled on Submission

5.3 Reports

Table 5.1: Reports Implemented

Report	Type	Based on Form	Purpose / Columns
All Buildings	List (default)	Buildings	Building ID, name, block, department, floors, capacity, status
All Energy Consumptions	List (default)	Energy_Consumption	Record ID, building, date, meter reading, kWh, cost, CO_2, status, CO_2 prediction
All Water Consumptions	List (default)	Water_Consumption	Record ID, building, date, litres, source, cost

Report	Type	Based on Form	Purpose / Columns
All Waste Management	List (default)	Waste_Management	Record ID, building, date, type, total, recycled, disposal method, recycling %
All Green Activities	List (default)	Green_Activities	Activity details, participants, trees planted, waste collected
All Sustainability Complaints	List (default)	Sustainability_Complaints	Complaint details, priority, assignee, status, resolution
All Student Participations	List (default)	Student_Participation	Student, activity, participation type, green points
Energy Consumption Report	Kanban	Energy_Consumption	Cards grouped by building, showing kWh, cost, CO_2 and status
Waste Management Report	List	Waste_Management	Waste records for review and filtering
Water Consumption Report	Pivot chart	Water_Consumption	Water usage analysed by building, source and date
Green Activities Report	Pivot table	Green_Activities	Participants analysed by activity, type, department and building
Campus Sustainability AI Assistant Report	List (default)	Campus_Sustainability_AI_Assistant	Question, AI response and time

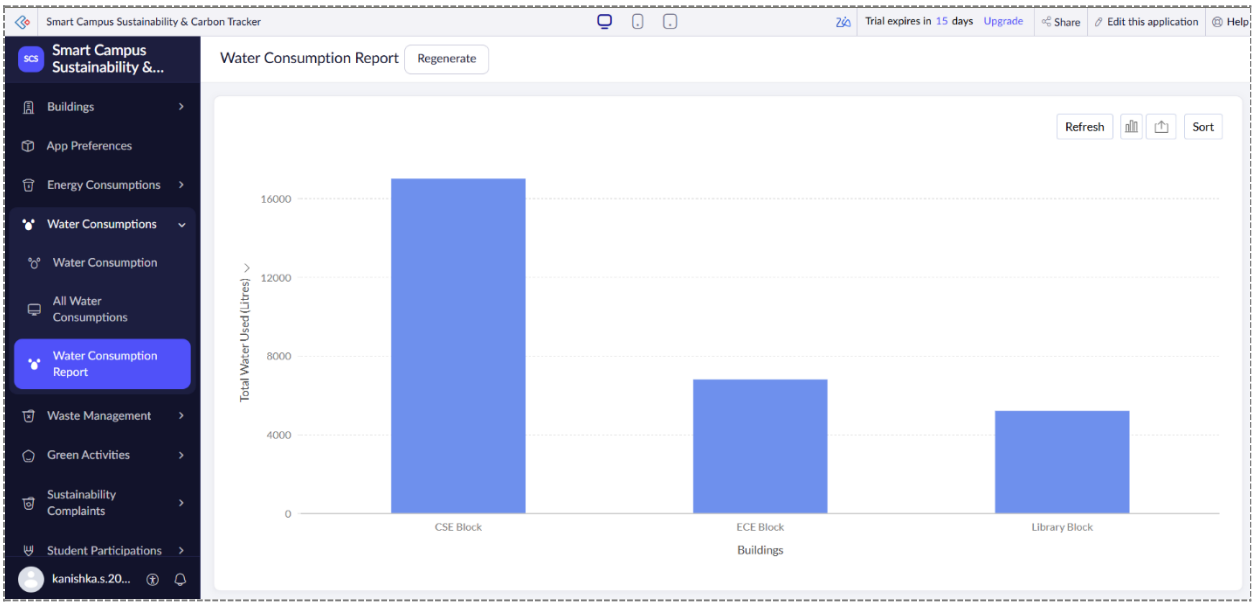


Figure 5.4: Water Pivot Chart Reports

5.4 Pages and Dashboards

The Sustainability Dashboard is a page built with the Zoho Creator page builder. It binds charts, a gauge and key-value panels directly to the forms, so it always shows current data. Table 5.2 lists its components.

Table 5.2: Components of the Sustainability Dashboard

Component	Type	Data Source	Measure
Chart	Column	Energy_Consumption by Building Name	Sum of Units Consumed (kWh)
Chart 1	Bar	Water_Consumption by Building Name	Sum of Water Used (Litres)
Chart 2	Pie	Waste_Management by Waste Type	Sum of Total Waste (kg)
Chart 3	Line	Energy_Consumption by Date	Sum of Units Consumed (kWh)
Chart 4	Column	Student_Participation by Activity Name	Sum of Number of Participants
Gauge	Gauge	Energy_Consumption	Average Estimated CO_2 (target 3000, maximum 5000)
KPI panels	Key values	Energy, Water, Waste, Green Activities	Total kWh, total litres, total waste (kg), total trees planted
Embedded reports (2)	Report	Existing reports	Detail view below the charts

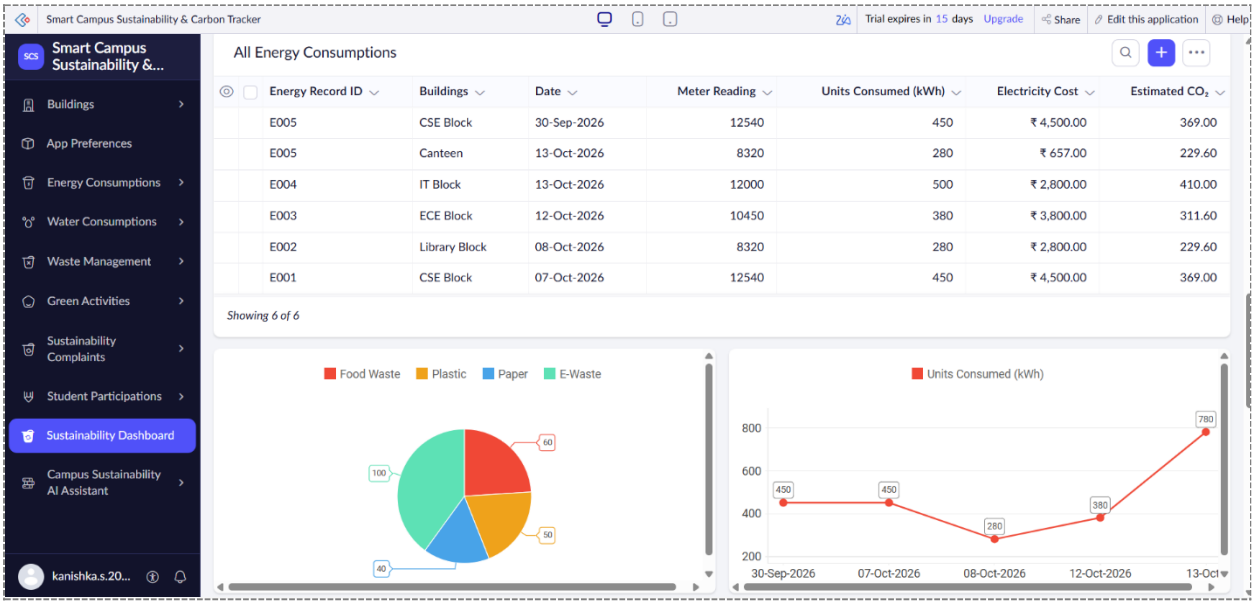


Figure 5.5: Sustainability Dashboard

5.5 Deluge Scripting

Deluge, Zoho's scripting language, is used for form validation workflows and for five reusable functions. The scripts are summarised in Table 5.3 and listed in full in Appendix C.

Table 5.3: Deluge Scripts Implemented

ID	Script	Type	Description
DS-01	Calculate_Recycling_Percentage	Form workflow	On add/edit (validate) of Waste Management: sets Recycling_Percentage = recycled ÷ total × 100, or 0 when total is 0

ID	Script	Type	Description
DS-02	Calculate_Estimated_CO2	Form workflow	On add/edit (validate) of Energy Consumption: sets Estimated_CO = units × 0.82, or 0
DS-03	High_Energy_Consumption_Alert	Form workflow	Shows an alert and cancels submission when units exceed 1000 kWh
DS-04	Energy_Status	Form workflow	Sets Energy Status to High Consumption above 1000 kWh, else Normal
DS-05	Campus_Sustainability_AI_Assistant	Form workflow	On add/edit (validate): stores askCampusAI answer and current time
DS-06	Display_AI_Answer	Form workflow	On add (validate): stores getOpenAIAnswer answer and current time
DS-07	Send_Sustainability_Complaint_to_Zoho_CRM	Form workflow	On add (success): creates a Contact in Zoho CRM with complaint details
DS-08	askCampusAI(question)	Function	Rule-based assistant: aggregates energy data, then answers greetings, recommendations, record counts and energy/carbon analytics
DS-09	getOpenAIAnswer(question)	Function	Validates the question and delegates to askCampusAI
DS-10	getSustainabilityInsights()	Function	Totals energy and carbon, computes the average and counts records above 150% of the average, and returns a recommendation
DS-11	getTotalEnergy()	Function	Returns Energy_Consumption.sum(Units_Consumed_kWh)
DS-12	testOpenAIConnection()	Function	Calls the OpenAI models endpoint through the connection to confirm authorisation

The carbon estimate is the simplest and most important script (Code 5.1). It implements the activity-data-times-emission-factor method of the GHG Protocol [9]. The factor 0.82 is written directly in the script; Chapter 9 recommends moving it to a configurable setting and replacing it with the latest published grid factor [10].

Code 5.1: Calculate_Estimated_CO2 (form workflow, on validate)

```
if(input.Units Consumed kWh > 0)
{
    input.Estimated CO = input.Units Consumed kWh * 0.82;
}
else
{
    input.Estimated CO = 0;
}
```

Code 5.2: getTotalEnergy function

```
float getTotalEnergy()
{
    total energy = Energy Consumption.sum(Units Consumed kWh);
    return total energy;
}
```

The askCampusAI function loops over all energy records once to accumulate the total, carbon total, average, highest and lowest record and the number of records at or above 500 kWh. It then

inspects the lower-cased question in a fixed order: greeting, electricity-saving advice, water, waste, green activities, student participation, complaints, energy listings, energy and carbon analytics, a general overview and finally a fallback listing the supported questions.

5.6 Workflows and Automation

Table 5.4: Workflows Implemented

Workflow	Form	Trigger	Action
Calculate Recycling Percentage	Waste_Management	Add or edit, on validate	Custom Deluge script (DS-01)
Calculate Estimated CO2	Energy_Consumption	Add or edit, on validate	Custom Deluge script (DS-02)
High Energy Consumption Alert	Energy_Consumption	Add or edit, on validate	Alert and cancel submit (DS-03)
Energy Status	Energy_Consumption	Add or edit, on validate	Set status field (DS-04)
Campus Sustainability AI Assistant	Campus_Sustainability_AI_Assistant	Add or edit, on validate	Call askCampusAI (DS-05)
Display AI Answer	Campus_Sustainability_AI_Assistant	Add, on validate	Call getOpenAIAnswer (DS-06)
Direct to – Form Properties	Campus_Sustainability_AI_Assistant	Pre-success	Open the AI Assistant report in a new window
Send Sustainability Complaint to Zoho CRM	Sustainability_Complaints	Add, on success	Create CRM Contact (DS-07)

All eight workflows are form workflows. Validation-time workflows are used for calculations so that the computed values are stored together with the record in a single submission. No scheduled workflows, approval processes, Blueprints or batch workflows were configured in this version; they are proposed as future enhancements (for example, a monthly consumption summary and an approval flow for complaint closure).

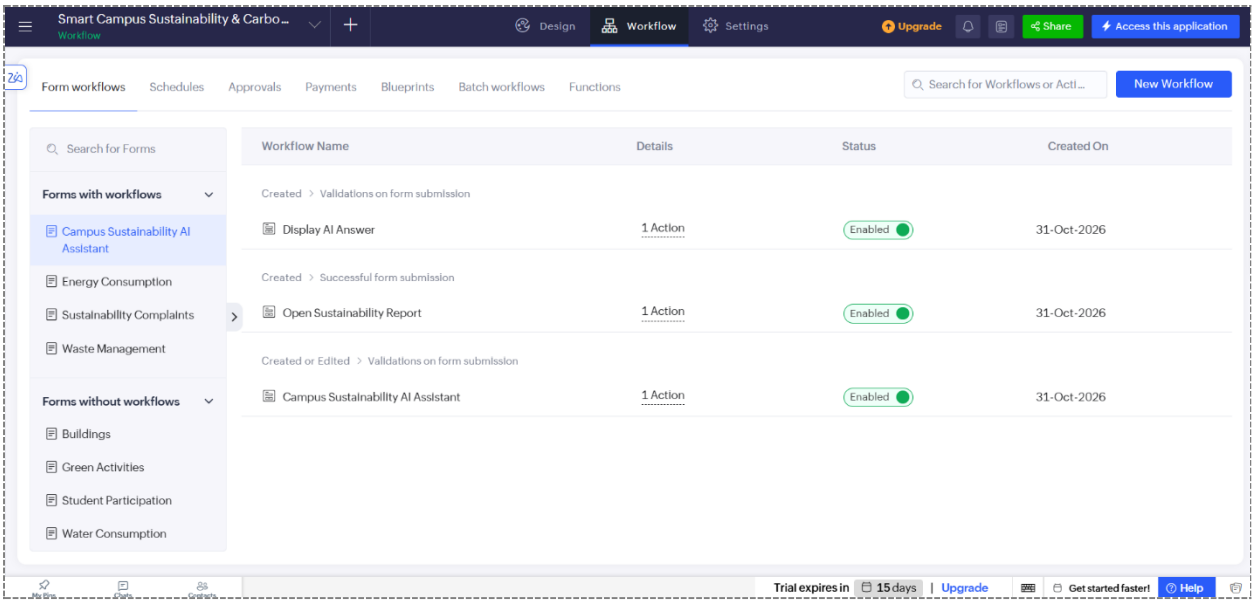


Figure 5.6: Workflow List of the Application

5.7 Integrations

Table 5.5: Integrations Implemented

Integration	Mechanism	Used By	Purpose
Zoho CRM	Deluge task zoho.crm.createRecord (Contacts module)	DS-07	Forwards each new complaint as a CRM contact with its details in the description
OpenAI connector	Custom connector openai_sustainability (bearer token) and connection openai_sustainability_connection; variable OpenAI_Model = gpt-4o-mini	DS-12	Confirms that the external AI service can be reached through an authorised connection
Zia prediction	AI field on Energy_Consumption (model Energy_Consumption_Model)	Estimated_CO_Prediction	Predicts estimated carbon from cost, meter reading and units

Code 5.3 shows the connection test. It sends a GET request to the OpenAI models endpoint using the stored connection, so no key appears in the script. In the current version the assistant's answers are produced by the rule-based askCampusAI function rather than by a language model; the connection is the foundation for adding model-generated answers later.

Code 5.3: testOpenAIConnection function

```
string testOpenAIConnection()
{
    response = invokeurl
    [
    url :"https://api.openai.com/v1/models"
    type :GET
    connection:"openai_sustainability_connection"
```

```
    ];
    return response.toString();
}
```

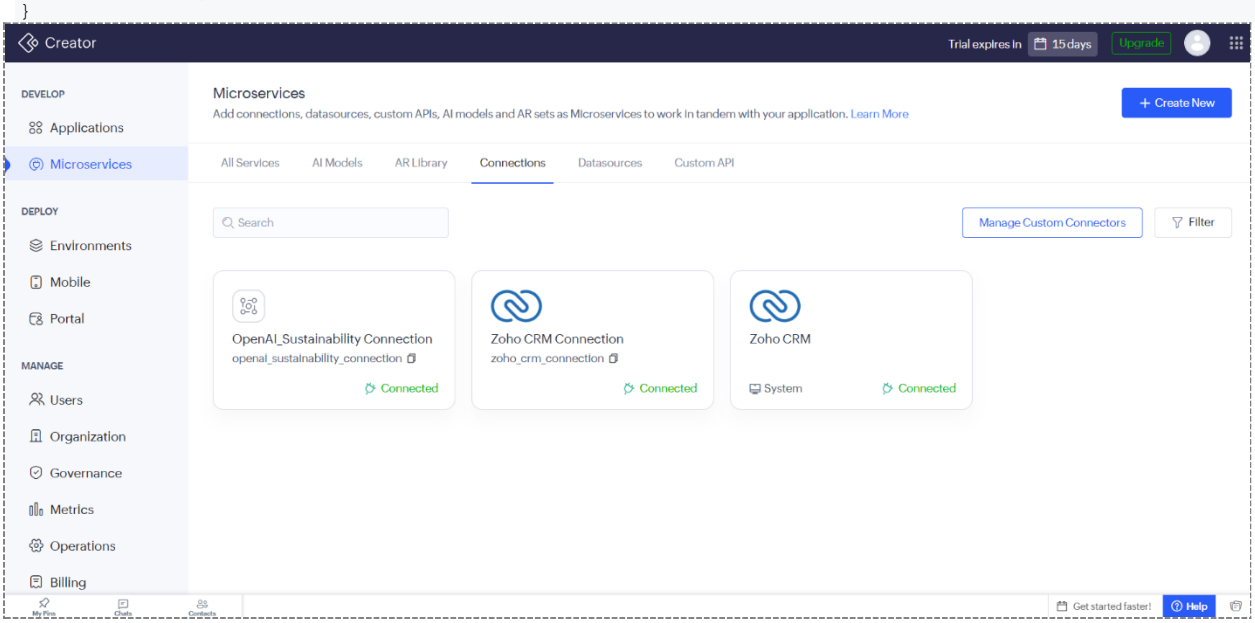


Figure 5.7: Connection and Integration Settings

5.8 User Management and Role-Based Access

The application relies on the profiles that Zoho Creator provides: Administrator, Write, Read, Developer and Customer, together with the default CEO role in the hierarchy. Users are invited and assigned a profile; the profile determines whether they can add, edit or view each component. The e-mail field of the Sustainability Complaints form is flagged as personal data, which marks it for special handling under the platform's data-protection features. Custom permission sets and record-level criteria were not created in this version (see Section 9.2).

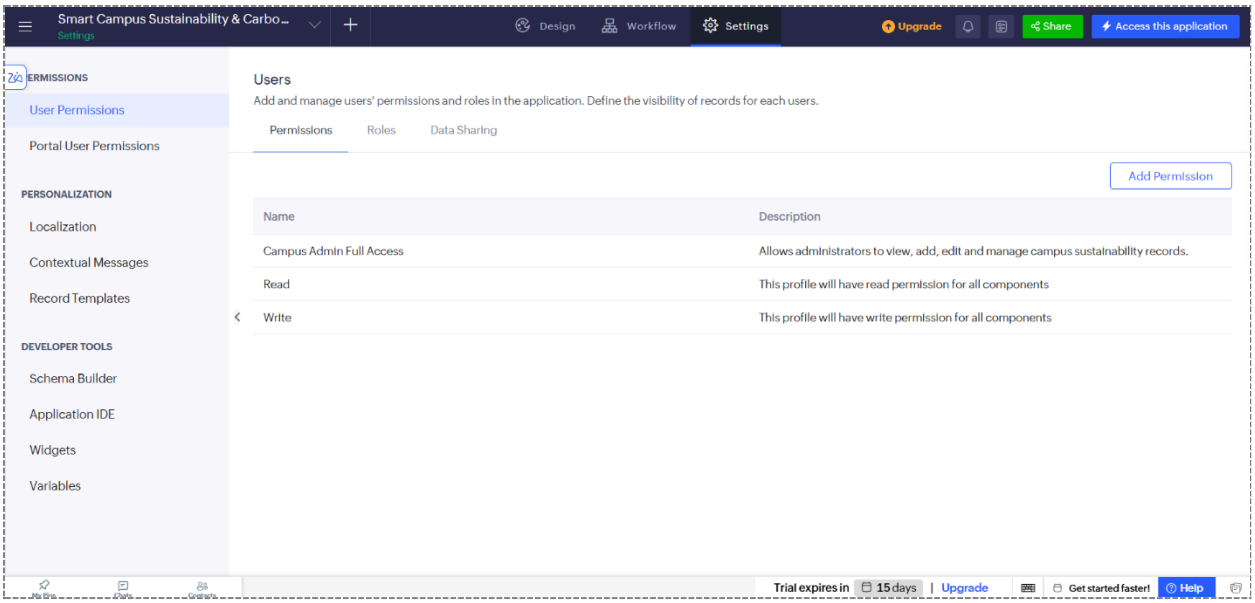


Figure 5.8: Profiles and Roles of the Application

5.9 AI and Analytics Features

Three features provide intelligence. First, the Estimated CO_2 Prediction field is a Zia AI field of type Prediction; it is trained from the existing records using cost, meter reading and units as supporting fields and shows a predicted carbon value alongside the formula-based estimate. Second, the Campus Sustainability AI Assistant lets a user type a question and receive an answer computed from live data, for example the highest or lowest consumption record, the average consumption, total estimated carbon or practical recommendations for saving electricity and water (Figure 5.9). Third, the dashboard and pivot reports provide analytics through charts, a gauge and key totals. The getSustainabilityInsights function adds a simple analytic rule: records above 150% of the average energy are counted as high usage and trigger a recommendation to inspect lighting, air conditioning and equipment.

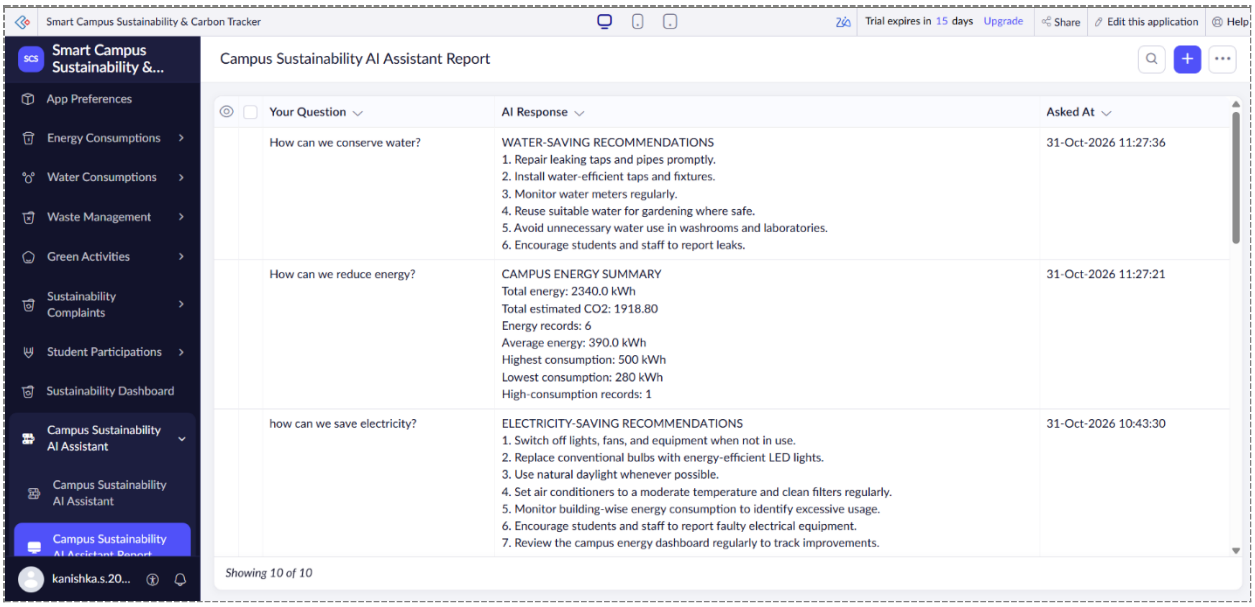


Figure 5.9: Campus Sustainability AI Assistant – Question and Response

5.10 Pro-Code Extension

Where visual configuration was insufficient, the application is extended with code. The Deluge functions perform aggregation and text analysis that no standard report provides; the invokeurl task calls an external REST API through an authorised connection; and the CRM task integrates a second Zoho product from within a form workflow. No separate programming stack, web server or custom widget was needed, which shows how far a single low-code platform can be extended with a small amount of script.

5.11 Mobile and Portal Access

The same application can be opened on the Zoho Creator mobile app for Android and iOS. Forms render as native screens, which allows facility staff to enter meter readings on site and students to submit complaints with a phone. A Customer (portal) profile exists with add and view permission and can be used to give outside users restricted access in the future.

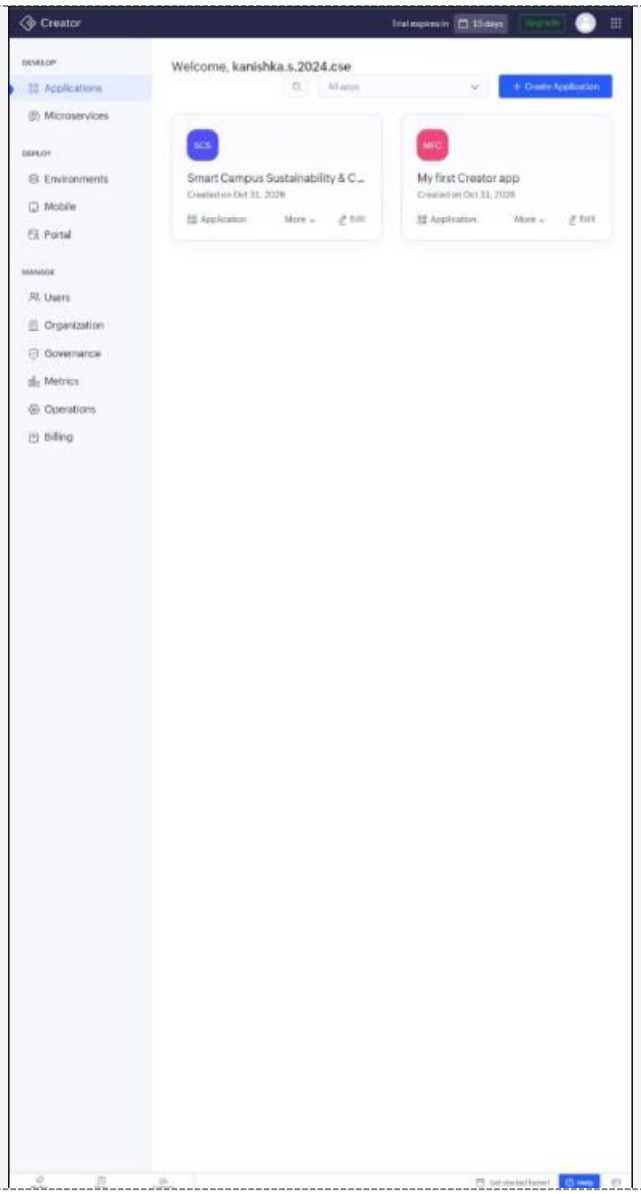


Figure 5.10: Application on the Zoho Creator Mobile App

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing is organised at four levels. Unit testing checks each form and workflow in isolation: every workflow is triggered from live mode, and each function is run with the Execute option of the function editor. Integration testing covers paths that cross components: a complaint submitted in the form must create a CRM contact; a question submitted in the AI Assistant form must store an answer; records added in the data forms must appear in the reports and change the dashboard totals. Access testing logs in with users of different profiles and checks what each can see and do. User acceptance testing is described in Section 6.5. Every test case is traced to a functional requirement in Table 6.1.

The "Status" column of Table 6.1 is intentionally left for the tester to complete (Pass or Fail) while the test is executed in live mode, and the actual values should be attached as screenshots.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Status (Pass / Fail)
TC-01	FR-01	Add a building with all mandatory fields	Enter ID, name, block, department, status; submit	Record saved; "Building Added Successfully" shown	
TC-02	FR-01	Mandatory validation on Buildings	Leave Building Name empty and submit	Form rejects the submission and highlights the field	
TC-03	FR-03	Carbon estimate for normal consumption	Energy record with Units = 500	Estimated $CO_2 = 500 \times 0.82 = 410$	
TC-04	FR-03	Carbon estimate with zero units	Energy record with Units = 0	Estimated $CO_2 = 0$	
TC-05	FR-04	Status for consumption at the limit	Units = 1000	Energy Status = Normal; record saved	
TC-06	FR-04	High consumption alert	Units = 1200	Alert "High Energy Consumption!" shown and submission cancelled	
TC-07	FR-02	Energy record lookup	Select a building from the Buildings lookup	Only existing buildings are offered, sorted ascending	
TC-08	FR-05	Add water record	Building, date, 15000 litres, source Municipal, cost	Record saved and visible in All Water Consumptions	
TC-09	FR-06	Recycling percentage	Total = 100 kg, Recycled = 40 kg	Recycling Percentage = 40	

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Status (Pass / Fail)
TC-10	FR-06	Recycling percentage with zero total	Total = 0, Recycled = 0	Recycling Percentage = 0 (no error)	
TC-11	FR-07	Add green activity	Type Tree Plantation, 25 participants, 50 trees, waste 5 kg	Record saved; trees counted in dashboard total	
TC-12	FR-08	Add student participation	Select a green activity; type Volunteer; 10 points	Record saved and visible in All Student Participations	
TC-13	FR-09	Add complaint	Complete all mandatory fields	Record saved; contact created in Zoho CRM	
TC-14	FR-09	Invalid e-mail	Enter “abc” in Email	Form rejects the e-mail value	
TC-15	FR-10	Kanban grouping	Open Energy Consumption Report	Cards grouped by building	
TC-16	FR-10	Pivot reports	Open Water Consumption Report and Green Activities Report	Pivot chart and pivot table render with current data	
TC-17	FR-11	Dashboard refresh	Add an energy record and reopen the dashboard	Column chart, line chart, gauge and kWh total reflect the new record	
TC-18	FR-12	Assistant greeting	Ask “hello”	Welcome message with supported topics	
TC-19	FR-12	Assistant recommendation	Ask “How can we save electricity?”	Numbered electricity-saving recommendations	
TC-20	FR-12	Assistant analytics	Ask “What is the highest energy consumption?”	Highest kWh value and its record ID	
TC-21	FR-12	Empty question	Submit an empty question	Answer “Please enter a question.”	
TC-22	FR-12	Unsupported question	Ask “weather tomorrow”	Fallback message listing example questions	
TC-23	FR-14	getSustainabilityInsights	Execute the function	Totals, average, high-usage count and a recommendation returned	
TC-24	FR-13	Zia prediction	Add several energy records and view the prediction field	Estimated CO_2 Prediction shows a value for new records	

6.3 Debugging Deluge Scripts

Deluge scripts were debugged with four techniques. First, the info statement writes a value to the execution log; the CRM workflow uses info crm_response to display the API response so a failed contact creation can be diagnosed. Second, functions can be executed directly from the function editor with sample arguments, which shows the returned string without submitting a form. Third,

form workflows can be tested in live mode, where an alert or the saved record reveals whether the logic ran. Fourth, the application's execution logs show which workflow ran, with the time and any error message.

Typical debugging steps were: confirm the field link names used in the script (for example Units_Consumed_kWh rather than the display name), confirm that the correct data type is used in arithmetic (number versus string), confirm that the trigger is the intended one (on validate versus on success), and check that the connection used by invokeurl is authorised.

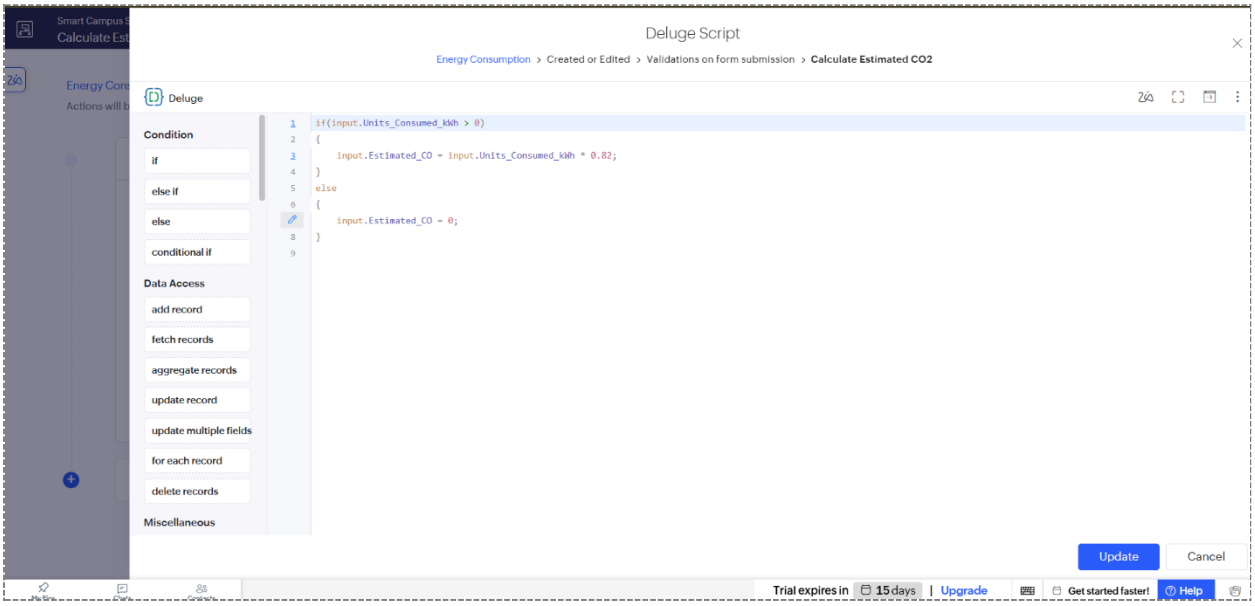


Figure 6.1: Running a Function with Execute in the Deluge Editor

6.4 Defect Log and Review Findings

A review of the exported application (version 1.0) against the test cases identified the points in Table 6.2. They are recorded honestly as open items; none of them stops the core carbon, recycling and dashboard features from working, and each has a proposed fix.

Table 6.2: Defect Log and Review Findings

ID	Finding	Component	Severity	Proposed Fix
BUG-01	Records above 1000 kWh are blocked by the alert workflow, so the High Consumption status can never be saved.	Energy workflows DS-03 / DS-04	High	Replace "cancel submit" with a warning message so that high records are saved with the status High Consumption.
BUG-02	The assistant counts a high-consumption record at 500 kWh while the form uses 1000 kWh.	askCampusAI (DS-08)	Medium	Use one shared threshold stored in an application variable.

ID	Finding	Component	Severity	Proposed Fix
BUG-03	Water, waste, activity, participation and complaint answers only count records and show a placeholder note about field names.	askCampusAI (DS-08)	Medium	Use aggregate functions on Water_Used_Litres, Total_Waste_kg and Status to return real totals.
BUG-04	Some dashboard panel captions still show template text (“New employees this month”, “All requests”, “Deals won”, “Open tickets today”).	Sustainability Dashboard	Medium	Rename the captions to Total Waste (kg), Trees Planted, Total Water (Litres) and Total Energy (kWh).
BUG-05	The redirect URL after the AI form begins with “http://https://”, which is malformed.	Direct to – Form Properties	Medium	Remove the extra “http://” prefix.
BUG-06	getOpenAIAnswer only delegates to askCampusAI; no language model is called.	DS-09	Low	Call the OpenAI connection with invokeurl and fall back to askCampusAI on failure.
BUG-07	The emission factor 0.82 is written inside the script.	DS-02	Medium	Store it as an application variable and update it from the latest published grid factor.
BUG-08	Recycled quantity can exceed total quantity and record IDs are not checked for duplicates.	Waste and other forms	Low	Add validation rules for quantity and unique-ID checks.
BUG-09	Complaint location is passed to CRM as a lookup identifier rather than a building name.	DS-07	Low	Read the building name from the Buildings form before sending.

6.5 User Acceptance Testing

User acceptance testing is performed by asking representative users (an administrator, a facility staff member and a student) to complete a short set of tasks: add a building, add an energy and a waste record, submit a complaint, open the dashboard and ask the assistant a question. Their comments are recorded in Table 6.3.

Table 6.3: User Acceptance Feedback (to be completed after the demonstration)

Tester Role	Task Performed	Comment	Rating (1–5)
Administrator	Review dashboard and reports		
Facility / Data Entry Staff	Add energy and waste records		
Student / Staff	Submit a complaint; ask the assistant		

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator provides Development, Stage and Production environments. Changes are made in Development, published to Stage for verification as a versioned release, and then published to Production, where end users work [6]. The deployment procedure is: open Environments under Deploy, choose Publish to Stage, select the application and its changed components, enter a title and description; after verifying the Stage version, choose Publish to Production. Published versions cannot be rolled back, so the rollback plan is to publish a corrected version and to keep a .ds export of every released version as a backup. A .ds export of this application (version 1.0, 21 October 2026) was taken for that purpose.

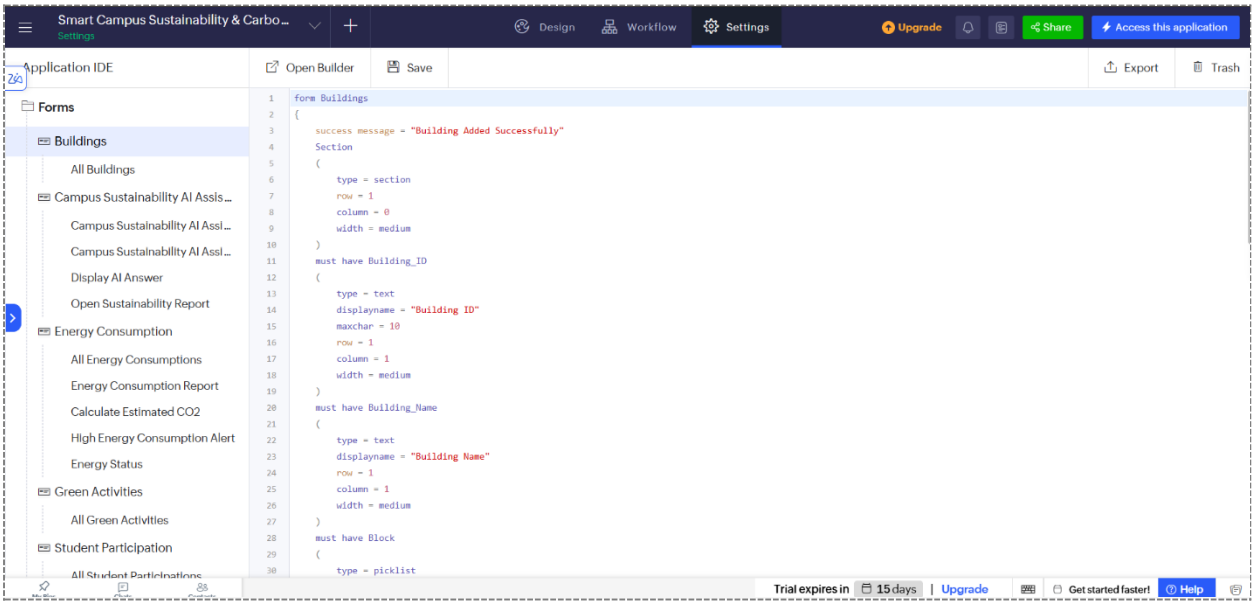


Figure 7.1: Environments – Publish to Stage and Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓ / ✗)
1	All test cases in Table 6.1 executed in Development / Stage	☐
2	Sample and test data removed or isolated	☐
3	Profiles, roles and users configured in Production	☐
4	OpenAI connection authorised and CRM integration enabled in Production	☐
5	Workflows enabled and verified	☐
6	Application published and URL verified	☐
7	Version / release notes recorded; .ds export stored	✓ (v1.0 export)

7.2 Performance Optimisation

The application is small, so optimisation focused on keeping work inside the platform's efficient paths. Table 7.2 lists the measures applied and the further measures recommended once data volumes grow. The measures are described in terms of processing approach rather than timings.

Table 7.2: Performance Optimisation Measures

Measure	Component	Effect	Status
Calculated values (CO_2, status, recycling %) are computed once on submission and stored	Validate workflows	Reports and dashboard read stored values; no recalculation at view time	Applied
Dashboard totals use built-in aggregate expressions (sum, average)	Sustainability Dashboard	Aggregation is done by the platform	Applied
Lookups to a single Buildings master instead of free-text names	All data forms	Consistent grouping and smaller records	Applied
Use aggregate functions such as Energy_Consumption.sum() instead of looping through every record	askCampusAI, getSustainabilityInsights	Faster answers as records grow	Recommended (getTotalEnergy already uses it)
Criteria on default reports (date range, building)	Reports	Smaller result sets	Recommended

7.3 Security Features

Table 7.3: Security Controls Implemented

Control	Implementation	Evidence
Authentication	Zoho account sign-in; only invited users access the application	Platform feature
Profile-based permissions	Administrator, Write, Read, Developer and Customer profiles	Section 5.8
Personal data flag	Email field of Sustainability Complaints marked as personal data	Form definition
Secured external access	OpenAI access through a bearer-token connector and connection; no key in the script	Section 5.7
Input validation	Mandatory fields, restricted drop-downs, e-mail type, numeric types and maximum lengths	Tables 4.2–4.9
Transport security	Application served over HTTPS by Zoho	Platform feature
Audit trail	System fields (added user and time) and execution logs	Section 7.4

7.4 Governance and Administration

Governance covers naming, ownership, change and monitoring. Forms use consistent link names (for example Units_Consumed_kWh and Total_Waste_kg), shared logic is kept in functions, and every release is exported as a .ds file. The administrator owns the emission factor and thresholds and should review them at least once a year when a new grid factor is published. Execution logs under Operations show workflow and function runs and are the first place to look when a record is not calculated or a CRM contact is missing. Data stewardship rests with the department that owns each form: facility staff for energy and water, housekeeping for waste, and student clubs for activities.

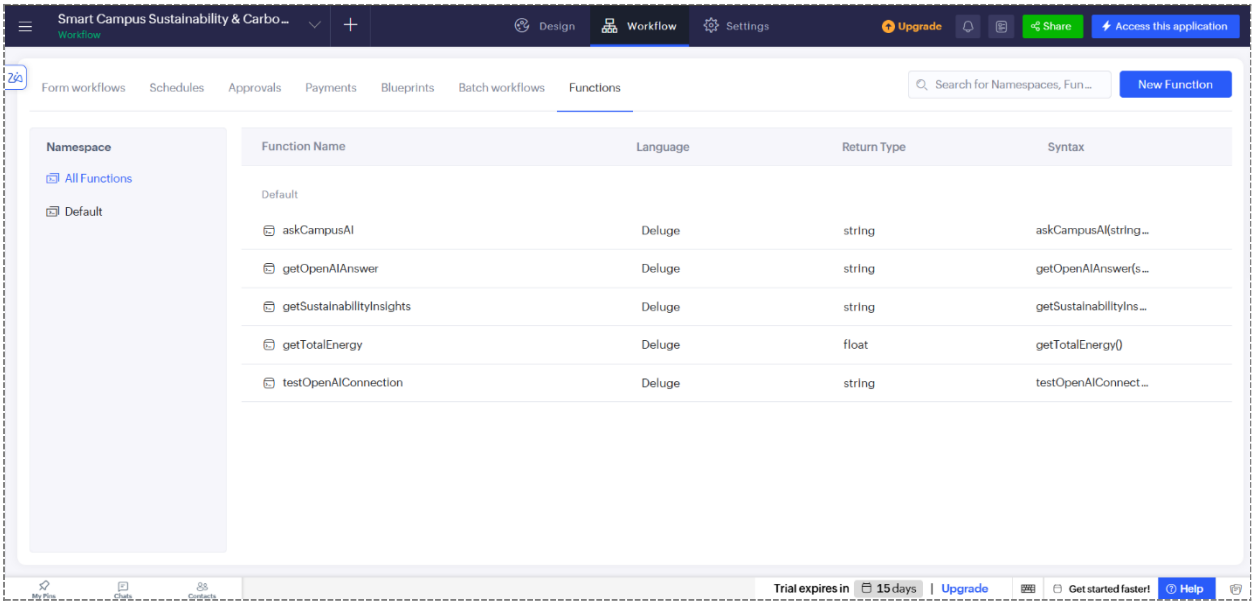


Figure 7.2: Application Logs

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The following scenario demonstrates the application end to end. The administrator adds the main campus buildings in the Buildings form. Facility staff then add an electricity record for a hostel, for example 500 kWh; on submission the application stores an estimated carbon value of 410 (500 × 0.82) and the status Normal. A waste record with 100 kg collected and 40 kg recycled shows a recycling percentage of 40. A student club logs a tree-plantation drive with 25 participants and 50 trees, and each participating student is recorded with a participation type and green points. A staff member reports a broken light; the complaint is stored and a contact appears in Zoho CRM.

The administrator opens the Sustainability Dashboard, where the energy column chart, water bar chart, waste pie chart, energy trend line, participation chart, carbon gauge and the totals for energy, water, waste and trees planted reflect the new records immediately. Finally, the administrator asks the assistant "What is the average energy consumption?" and receives the average kWh per record, the total energy and the number of records analysed.

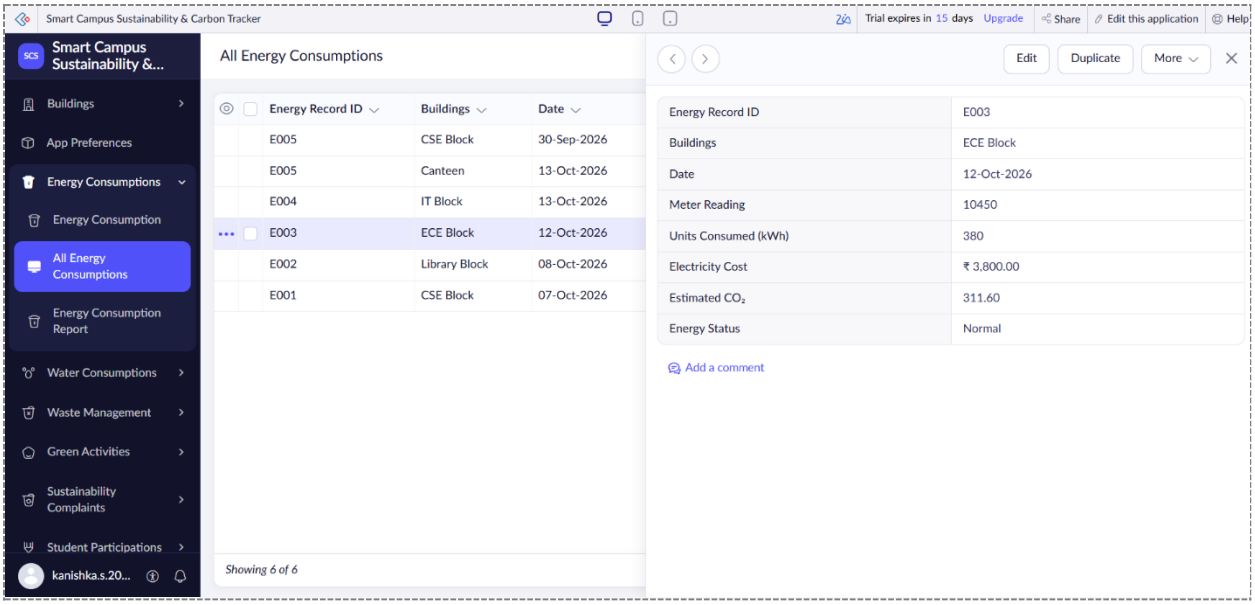


Figure 8.1: Scenario Step 1 – Energy Record Saved with Estimated CO_2

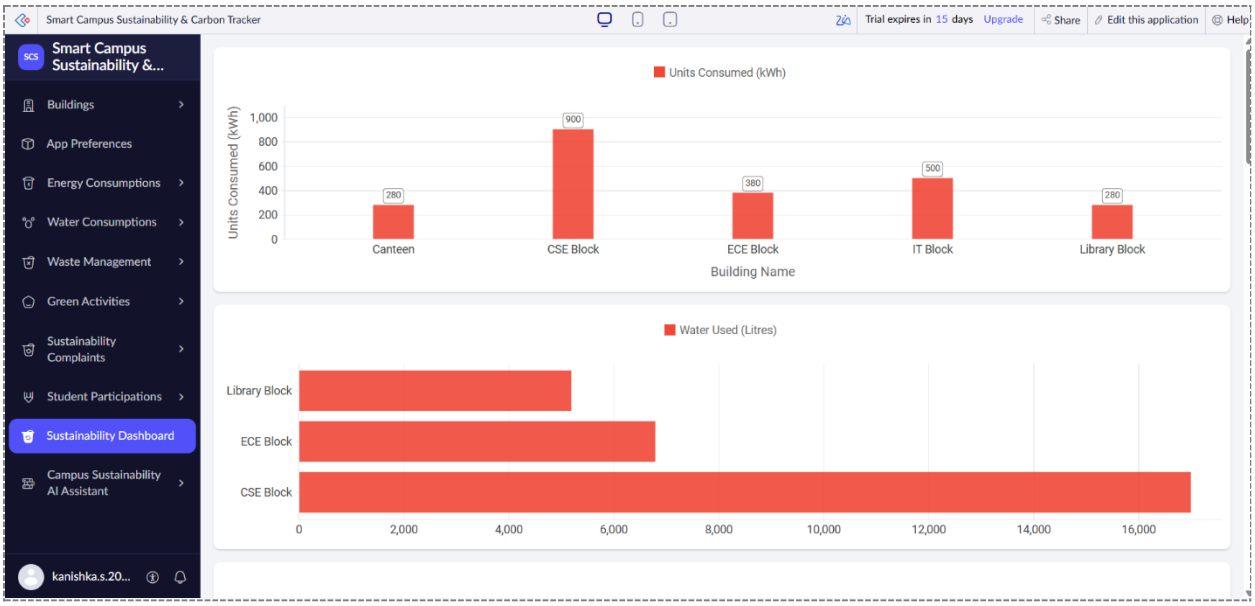


Figure 8.2: Scenario Step 2 – Dashboard Updated

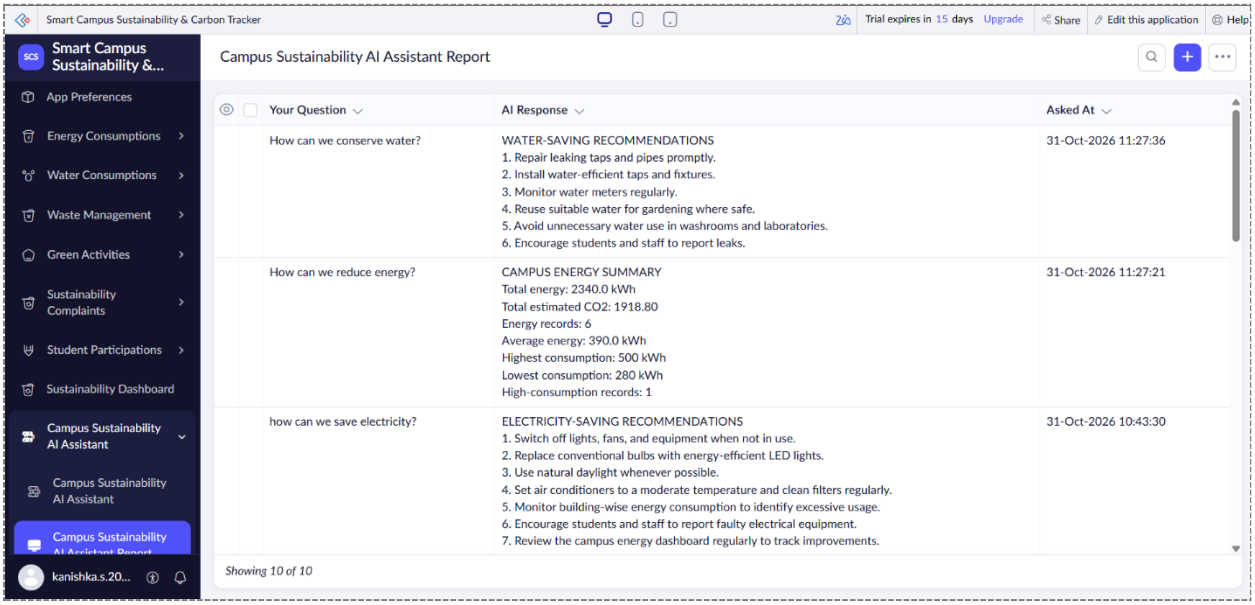


Figure 8.3: Scenario Step 3 – AI Assistant Response

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective	Outcome	Evidence
Data model and forms for all sustainability subjects	Eight linked forms with lookups and controlled drop-downs	Tables 4.1–4.9
Automatic carbon estimate and high-consumption flag	Estimated CO_2 = kWh × factor; status and alert workflows	DS-02 to DS-04
Automatic recycling percentage	Computed on every waste record	DS-01

Objective	Outcome	Evidence
Reports and dashboard	Twelve reports and a dashboard with five charts, a gauge and KPI panels	Tables 5.1, 5.2
AI Assistant	Rule-based assistant answering energy, carbon and recommendation questions	DS-05, DS-08
Complaint integration and AI prediction	Zoho CRM contact creation and Zia prediction field	Section 5.7

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Aspect	Existing System	Proposed Application
Data storage	Registers and spreadsheets	Single Zoho Creator database with lookups
Carbon calculation	Not done or manual	Automatic on every energy record
Recycling performance	Not measured	Percentage calculated per waste record
Complaint handling	Informal messages	Structured register with priority, status and CRM contact
Student participation	Not recorded	Participation type and green points stored per activity
Management view	Manual report at term end	Live dashboard and reports
Answering questions	Manual filtering	AI Assistant answers from stored data

8.4 Limitations of the Low-Code Approach

- The page builder offers a fixed set of chart types and styling; unusual visualisations would need a custom widget.
- Business rules written in scripts are not visible in the data model, so they must be documented, as the inconsistent thresholds in Table 6.2 show.
- Aggregations written as loops can become slow when records grow, and some aggregate behaviour in Deluge differs from what the documentation suggests.
- The platform is subscription based and the application depends on Zoho's hosting and release cycle.
- The rule-based assistant understands only the keywords it was programmed with; it is not a general conversational model.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The forms, reports and dashboard were produced without code, using the no-code end of the spectrum. The calculations and validations needed small Deluge scripts, which is the low-code

middle. The analytics assistant, the REST call through a connection and the CRM integration sit toward the pro-code end, where understanding of data structures, APIs and debugging matters. The project shows that most of the application could be delivered visually, and that targeted code adds the behaviour that makes it useful.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project built the Smart Campus Sustainability & Carbon Tracker, a Zoho Creator application that gathers campus buildings, electricity, water, waste, green activities, student participation and complaints into one system. It estimates carbon emissions for every electricity record, calculates recycling percentages, summarises the data on a dashboard and in reports, forwards complaints to Zoho CRM and answers sustainability questions through an in-app assistant. The project used the major capabilities of the platform covered in the course: data modelling, forms, reports, pages, Deluge scripting, form workflows, integration, AI fields and profiles. It also showed where the platform reaches its limits and where careful review, as recorded in Table 6.2, improves the result.

9.2 Future Enhancements

- Replace the hard-coded emission factor with a configurable setting based on the latest published grid factor, and add factors for fuel and other sources.
- Fix the high-consumption logic (BUG-01) and use one shared threshold across the forms and the assistant.
- Add scheduled workflows for monthly consumption summaries and e-mail or in-app notifications for high-consumption buildings.
- Add an approval process or Blueprint for closing complaints, and route complaints automatically to the responsible team.
- Add custom permission sets and record-level criteria so that each department sees only its own data.
- Connect the assistant to a language model through the configured OpenAI connection, with the rule-based function as fallback.
- Show trend comparisons (month on month and year on year) and building rankings on the dashboard.
- Integrate smart meters or IoT sensors so that readings flow into the application automatically.
- Extend carbon accounting to waste and travel emissions and add a leaderboard based on green points.

9.3 Learning Outcomes and Career Relevance

The project developed skills in modelling data with forms and lookups, writing and debugging Deluge, building dashboards, calling external services through connections and reviewing an application critically against its own test cases. It also showed the importance of documenting thresholds and constants. These skills map to roles such as citizen developer, low-code developer, Zoho Creator consultant and automation engineer, and the experience of extending the platform with code is a step towards professional software development.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help.” [Online]. Available: https://www.zoho.com/creator/help/

[2] A. C. Bock and U. Frank, “Low-code platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[3] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” Software and Systems Modeling, vol. 21, pp. 437–446, 2022, doi: 10.1007/s10270-021-00970-2.

[4] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/

[5] Zoho Corporation, “Zoho CRM Developer Documentation (API).” [Online]. Available: https://www.zoho.com/crm/developer/docs/

[6] Zoho Corporation, “Environments and publishing applications to Stage and Production,” Zoho Creator Help. [Online]. Available: https://www.zoho.com/creator/help/

[7] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178.

[8] M. A. A. Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider, and A. Iqbal, “An empirical study of developer discussions on low-code software development challenges,” arXiv:2103.11429, 2021.

[9] World Resources Institute and World Business Council for Sustainable Development, “The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard, Revised Edition,” 2004. [Online]. Available: https://ghgprotocol.org

[10] Central Electricity Authority, Ministry of Power, Government of India, “CO_2 Baseline Database for the Indian Power Sector.” [Online]. Available: https://cea.nic.in

[11] Universitas Indonesia, “UI GreenMetric World University Rankings.” [Online]. Available: https://greenmetric.ui.ac.id

[12] Zoho Corporation, “AI fields (Prediction) in Zoho Creator,” Zoho Creator Help. [Online]. Available: https://www.zoho.com/creator/help/

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Menu sections for each subject, App Preferences, Approvals	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Text, number, decimal, date, date-time, e-mail, multi-line, drop-down, lookup, currency, AI prediction	5.2	Tables 4.2–4.9
II	Lookup field (relationship between forms)	Y	Buildings and Green Activities lookups	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not used in this version	—	—
II	Field validations / layout rules	Y	Mandatory fields, drop-downs, validate workflows	5.2, 5.5	DS-01 to DS-04
II	List report with filter, grouping, custom action	Y	Nine list reports	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Energy Consumption Report (kanban)	5.3	Fig. 5.4
II	Pivot table / chart report	Y	Water Consumption Report (pivot chart); Green Activities Report (pivot table)	5.3	Table 5.1
II	Page / dashboard built with UI Builder	Y	Sustainability Dashboard	5.4	Fig. 5.5
II	Deluge – form events (validate / success)	Y	Seven custom scripts and one redirect action	5.5	Appendix C
II	Deluge – functions, fetch, loops, collections	Y	askCampusAI, getSustainabilityInsights and three more	5.5	Appendix C
II	Email / in-app notifications	N	Not configured; proposed as future work	9.2	—
II	Form workflow	Y	8 form workflows	5.6	Table 5.4
II	Scheduled workflow	N	Not configured; proposed as future work	9.2	—
II	Approval process	N	Not configured; system Approvals section only	9.2	—
II	Blueprint	N	Not configured	9.2	—
II	Batch / report workflow	N	Not configured	9.2	—
II	Testing & debugging evidence	Y	24 test cases; 9 logged findings	6	Tables 6.1, 6.2
III	Integration with a Zoho service	Y	Zoho CRM contact creation	5.7	Table 5.5
III	Connection / REST API call (invokeurl)	Y	openai_sustainability_connection	5.7	Code 5.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Webhook or Creator REST API	N	Not used	—	—
III	Users, roles and permission profiles	Y	Built-in profiles and CEO role	5.8	Fig. 5.8, Table 4.10
III	Record-level access criteria / customer portal	N	Customer profile available; no custom criteria	5.8, 9.2	—
III	Zia / AI feature	Y	Prediction field on Energy Consumption	5.9	Fig. 5.9
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts, gauge and KPI panels; pivot reports	5.4, 5.9	Fig. 5.5
IV	Performance optimisation applied & measured	Y	Stored calculations; aggregate expressions	7.2	Table 7.2
IV	Deployment across environments	Y	Procedure and .ds export	7.1	Fig. 7.1
IV	Security & governance	Y	Profiles, personal-data flag, connection, logs	7.3, 7.4	Table 7.3
V	Pro-code extension	Y	Deluge functions and REST/CRM calls	5.10	Code 5.3
V	Evaluation of low-code limits	Y	Limitations and reflection	8.4, 8.5	—

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The scripts below are reproduced from the exported application (version 1.0). Scripts DS-01 to DS-07 are form workflows and DS-08 to DS-12 are functions.

C.1: Calculate_Recycling_Percentage (DS-01)

```
if(input.Total Waste kg > 0)
{
    input.Recycling Percentage = input.Recycled Quantity kg / input.Total Waste kg * 100;
}
else
{
    input.Recycling Percentage = 0;
}
```

C.2: Calculate_Estimated_CO2 (DS-02)

```
if(input.Units Consumed kWh > 0)
{
    input.Estimated CO = input.Units Consumed kWh * 0.82;
}
else
{
    input.Estimated CO = 0;
}
```

C.3: High_Energy_Consumption_Alert (DS-03)

```
if(input.Units Consumed kWh > 1000)
{
    alert "High Energy Consumption! Please review this building's energy usage.";
    cancel submit;
}
```

C.4: Energy_Status (DS-04)

```
if(input.Units Consumed kWh > 1000)
{
    input.Energy Status = "High Consumption";
}
else
{
    input.Energy Status = "Normal";
}
```

C.5: Campus_Sustainability_AI_Assistant (DS-05)

```
questionText = ifnull(input.User Question,"");
if(questionText.trim() != "")
{
    input.Assistant Answer = thisapp.askCampusAI(questionText);
}
else
{
    input.Assistant Answer = "Please enter a question.";
}
input.Asked_At = zoho.currenttime;
```

C.6: Display_AI_Answer (DS-06)

```
questionText = ifnull(input.User Question,"");
if(questionText.trim() != "")
{
    input.Assistant Answer = thisapp.getOpenAIAnswer(questionText);
}
else
{
    input.Assistant Answer = "Please enter a question.";
}
input.Asked_At = zoho.currenttime;
```

C.7: Send_Sustainability_Complaint_to_Zoho_CRM (DS-07)

```
crm response =
zoho.crm.createRecord("Contacts",{"Last Name":input.Reported By,"Email":input.Email,"Description
":"Complaint ID: " + input.Complaint_ID + "\nBuilding/Location: " + input.Building_Location +
```

```
"\nCategory: " + input.Complaint_Category + "\nDescription: " + input.Description + "\nPriority:
" + input.Priority + "\nStatus: " + input.Status},Map());
info crm_response;
```

C.8: Direct_to_Form_Properties (pre-success action, as exported)

```
openURL("http://https://creatorapp.zoho.in/kanishka.s.2024.cse rajal703/smart-campus-
sustainability-carbon-tracker/#Report:Campus_Sustainability_AI_Assistant_Report","new window");
```

C.9: askCampusAI (DS-08)

```
string askCampusAI(string question)
{
    if(question == null || question.trim() == "")
    {
        return "Please enter a sustainability question.";
    }
    q = question.toLowerCase().trim();
    // ====================================
    // 1. ENERGY ANALYTICS
    // ====================================
    energyTotal = 0.0;
    co2Total = 0.0;
    energyCount = 0;
    highCount = 0;
    highestEnergy = 0.0;
    lowestEnergy = 0.0;
    highestID = "";
    lowestID = "";
    energyDetails = "";
    for each  rec in Energy_Consumption[ID != 0]
    {
        units = ifnull(rec.Units_Consumed_kWh,0.0);
        co2 = ifnull(rec.Estimated_CO,0.0);
        energyCount = energyCount + 1;
        energyTotal = energyTotal + units;
        co2Total = co2Total + co2;
        if(energyCount == 1)
        {
            highestEnergy = units;
            lowestEnergy = units;
            highestID = rec.ID.toString();
            lowestID = rec.ID.toString();
        }
        else
        {
            if(units > highestEnergy)
            {
                highestEnergy = units;
                highestID = rec.ID.toString();
            }
            if(units < lowestEnergy)
            {
                lowestEnergy = units;
                lowestID = rec.ID.toString();
            }
        }
        if(units >= 500)
        {
            highCount = highCount + 1;
        }
        energyDetails = energyDetails + "\nRecord ID: " + rec.ID.toString() + "\nEnergy: " +
units.toString() + " kWh" + "\nEstimated CO2: " + co2.toString() + "\n----------------------";
    }
    energyAverage = 0.0;
    if(energyCount > 0)
    {
        energyAverage = energyTotal / energyCount;
    }
    // ====================================
    // 2. GREETING
    // ====================================
    if(q == "hi" || q == "hello" || q == "hey" || q == "good morning" || q == "good afternoon")
    {
        return "Welcome to the Campus Sustainability Assistant!\n" + "Ask about electricity, energy,
carbon emissions, water, waste, " + "green activities, student participation, complaints, or
sustainability recommendations.";
    }
    // ====================================
    // 3. ELECTRICITY-SAVING RECOMMENDATIONS
    // ====================================
    if(q.contains("electricity") || q.contains("save power") || q.contains("saving power") ||
q.contains("electrical energy"))
    {
        if(q.contains("save") || q.contains("reduce") || q.contains("conserve") ||
q.contains("recommend") || q.contains("how can") || q.contains("suggest"))
```

```
        {
            return "ELECTRICITY-SAVING RECOMMENDATIONS\n" + "1. Switch off lights, fans, and equipment
when not in use.\n" + "2. Replace conventional bulbs with energy-efficient LED lights.\n" + "3. Use
natural daylight whenever possible.\n" + "4. Set air conditioners to a moderate temperature and clean
filters regularly.\n" + "5. Monitor building-wise energy consumption to identify excessive usage.\n" +
"6. Encourage students and staff to report faulty electrical equipment.\n" + "7. Review the campus
energy dashboard regularly to track improvements.";
        }
    }
    // ====================================
    // 4. WATER CONSUMPTION
    // ====================================
    if(q.contains("water"))
    {
        if(q.contains("recommend") || q.contains("reduce") || q.contains("save") ||
q.contains("conserve") || q.contains("how can") || q.contains("suggest"))
        {
            return "WATER-SAVING RECOMMENDATIONS\n" + "1. Repair leaking taps and pipes promptly.\n" +
"2. Install water-efficient taps and fixtures.\n" + "3. Monitor water meters regularly.\n" + "4. Reuse
suitable water for gardening where safe.\n" + "5. Avoid unnecessary water use in washrooms and
laboratories.\n" + "6. Encourage students and staff to report leaks.";
        }
        waterCount = 0;
        waterDetails = "";
        for each  w in Water_Consumption[ID != 0]
        {
            waterCount = waterCount + 1;
            waterDetails = waterDetails + "\nWater record ID: " + w.ID.toString() + "\n----------
-------";
        }
        return "WATER CONSUMPTION RECORDS\n" + "Records found: " + waterCount + "\n" + waterDetails +
"\nWater quantity totals require your actual quantity field name.";
    }
    // ====================================
    // 5. WASTE MANAGEMENT
    // ====================================
    if(q.contains("waste") || q.contains("recycl") || q.contains("garbage") || q.contains("rubbish"))
    {
        if(q.contains("recommend") || q.contains("reduce") || q.contains("manage") ||
q.contains("suggest") || q.contains("how can") || q.contains("recycle"))
        {
            return "WASTE-REDUCTION RECOMMENDATIONS\n" + "1. Provide clearly labelled recycling
bins.\n" + "2. Reduce single-use plastics on campus.\n" + "3. Compost suitable food and garden
waste.\n" + "4. Encourage reusable bottles, bags, and containers.\n" + "5. Separate recyclable and
non-recyclable waste.\n" + "6. Track waste quantities to measure improvement.";
        }
        wasteCount = 0;
        wasteDetails = "";
        for each  ws in Waste_Management[ID != 0]
        {
            wasteCount = wasteCount + 1;
            wasteDetails = wasteDetails + "\nWaste record ID: " + ws.ID.toString() + "\n----------
--------";
        }
        return "WASTE MANAGEMENT RECORDS\n" + "Records found: " + wasteCount + "\n" + wasteDetails +
"\nWaste quantity totals require your actual quantity field name.";
    }
    // ====================================
    // 6. GREEN ACTIVITIES
    // ====================================
    if(q.contains("green activit") || q.contains("tree planting") || q.contains("plantation") ||
q.contains("cleanup") || q.contains("clean-up") || q.contains("environmental event"))
    {
        if(q.contains("recommend") || q.contains("suggest") || q.contains("idea") || q.contains("how
can"))
        {
            return "GREEN ACTIVITY IDEAS\n" + "1. Organize tree-planting events.\n" + "2. Conduct
campus cleanup drives.\n" + "3. Organize environmental awareness sessions.\n" + "4. Encourage reusable
bottles and bags.\n" + "5. Arrange campus recycling campaigns.\n" + "6. Invite students to participate
in sustainability projects.";
        }
        activityCount = 0;
        activityDetails = "";
        for each  ga in Green_Activities[ID != 0]
        {
            activityCount = activityCount + 1;
            activityDetails = activityDetails + "\nActivity record ID: " + ga.ID.toString() + "\n-----
-----------------";
        }
        return "GREEN ACTIVITIES\n" + "Activity records found: " + activityCount + "\n" +
activityDetails;
    }
    // ====================================
    // 7. STUDENT PARTICIPATION
    // ====================================
    if(q.contains("student") || q.contains("participation") || q.contains("participants") ||
q.contains("volunteer"))
```

```
    {
        participationCount = 0;
        participationDetails = "";
        for each  st in Student Participation[ID != 0]
        {
            participationCount = participationCount + 1;
            participationDetails = participationDetails + "\nParticipation record ID: " +
st.ID.toString() + "\n----------------------";
        }
        return "STUDENT PARTICIPATION\n" + "Participation records found: " + participationCount + "\n"
+ participationDetails + "\nThis counts records, not necessarily unique students.";
    }
    // ====================================
    // 8. SUSTAINABILITY COMPLAINTS
    // ====================================
    if(q.contains("complaint") || q.contains("pending") || q.contains("resolved") ||
q.contains("reported problem"))
    {
        complaintCount = 0;
        complaintDetails = "";
        for each  cp in Sustainability Complaints[ID != 0]
        {
            complaintCount = complaintCount + 1;
            complaintDetails = complaintDetails + "\nComplaint record ID: " + cp.ID.toString() + "\n--
--------------------";
        }
        return "SUSTAINABILITY COMPLAINTS\n" + "Total complaint records: " + complaintCount + "\n" +
complaintDetails + "\nPending and resolved totals require your actual status field name.";
    }
    // ====================================
    // 9. ENERGY RECORDS
    // ====================================
    if((q.contains("energy") && q.contains("all") || q.contains("records") || q.contains("list") ||
q.contains("show") || q.contains("display")) || q.contains("energy details"))
    {
        return "ENERGY CONSUMPTION RECORDS\n" + "Records found: " + energyCount + "\n" + "Total
energy: " + energyTotal + " kWh\n" + energyDetails;
    }
    // ====================================
    // 10. ENERGY AND CARBON ANALYTICS
    // ====================================
    if(q.contains("energy") || q.contains("electricity") || q.contains("consumption") ||
q.contains("kwh") || q.contains("carbon") || q.contains("co2") || q.contains("emission"))
    {
        if(q.contains("highest") || q.contains("maximum") || q.contains("most energy") ||
q.contains("peak"))
        {
            if(energyCount == 0)
            {
                return "No energy records are available for analysis.";
            }
            return "HIGHEST ENERGY CONSUMPTION\n" + "Energy: " + highestEnergy + " kWh\n" + "Record
ID: " + highestID;
        }
        if(q.contains("lowest") || q.contains("minimum") || q.contains("least energy"))
        {
            if(energyCount == 0)
            {
                return "No energy records are available for analysis.";
            }
            return "LOWEST ENERGY CONSUMPTION\n" + "Energy: " + lowestEnergy + " kWh\n" + "Record ID:
" + lowestID;
        }
        if(q.contains("average") || q.contains("mean"))
        {
            return "AVERAGE ENERGY CONSUMPTION\n" + "Average: " + energyAverage + " kWh per record\n"
+ "Total energy: " + energyTotal + " kWh\n" + "Records analyzed: " + energyCount;
        }
        if(q.contains("co2") || q.contains("carbon") || q.contains("emission"))
        {
            return "CARBON EMISSIONS SUMMARY\n" + "Total estimated CO2: " + co2Total + "\n" + "Records
analyzed: " + energyCount;
        }
        if(q.contains("high consumption") || q.contains("above 500") || q.contains("at least 500"))
        {
            return "HIGH-CONSUMPTION RECORDS\n" + "Records consuming at least 500 kWh: " + highCount;
        }
        return "CAMPUS ENERGY SUMMARY\n" + "Total energy: " + energyTotal + " kWh\n" + "Total
estimated CO2: " + co2Total + "\n" + "Energy records: " + energyCount + "\n" + "Average energy: " +
energyAverage + " kWh\n" + "Highest consumption: " + highestEnergy + " kWh\n" + "Lowest consumption: "
+ lowestEnergy + " kWh\n" + "High-consumption records: " + highCount;
    }
    // ====================================
    // 11. GENERAL SUSTAINABILITY
    // ====================================
    if(q.contains("sustainability") || q.contains("environment") || q.contains("eco-friendly") ||
q.contains("green campus"))
```

```
    {
        return "CAMPUS SUSTAINABILITY OVERVIEW\n" + "Total energy: " + energyTotal + " kWh\n" +
"Estimated CO2: " + co2Total + "\n" + "Energy records: " + energyCount + "\n\n" + "You can ask about
electricity conservation, water saving, " + "waste reduction, green activities, student participation,
" + "or sustainability complaints.";
    }
    // ====================================
    // 12. GENERAL FALLBACK
    // ====================================
    return "I could not match that question to a supported topic.\n" + "Try asking:\n" + "- How can we
save electricity?\n" + "- How can we reduce carbon emissions?\n" + "- How can we save water?\n" + "-
How can we reduce waste?\n" + "- Suggest green campus activities.\n" + "- Show energy consumption
records.\n" + "- What is the average energy consumption?";
}
```

C.10: getOpenAIAnswer (DS-09)

```
string getOpenAIAnswer(string question)
{
    if(question == null || question.trim() == "")
    {
        return "Please enter a sustainability question.";
    }
    return thisapp.askCampusAI(question);
}
```

C.11: getSustainabilityInsights (DS-10)

```
string getSustainabilityInsights()
{
    totalEnergy = 0.0;
    totalCO2 = 0.0;
    recordCount = 0;
    highUsageCount = 0;
    for each  rec in Energy_Consumption[ID != 0]
    {
        energy = ifnull(rec.Units_Consumed_kWh,0.0);
        co2 = ifnull(rec.Estimated CO,0.0);
        totalEnergy = totalEnergy + energy;
        totalCO2 = totalCO2 + co2;
        recordCount = recordCount + 1;
    }
    averageEnergy = 0.0;
    if(recordCount > 0)
    {
        averageEnergy = totalEnergy / recordCount;
    }
    for each  rec in Energy_Consumption[ID != 0]
    {
        energy = ifnull(rec.Units Consumed kWh,0.0);
        if(averageEnergy > 0 && energy > averageEnergy * 1.5)
        {
            highUsageCount = highUsageCount + 1;
        }
    }
    result = "CAMPUS SUSTAINABILITY ANALYTICS\n\n";
    result = result + "Total energy consumption: " + totalEnergy + " kWh\n";
    result = result + "Total estimated CO2: " + totalCO2 + "\n";
    result = result + "Energy records analyzed: " + recordCount + "\n";
    result = result + "Average energy per record: " + averageEnergy + " kWh\n";
    result = result + "High-consumption records: " + highUsageCount + "\n\n";
    if(recordCount == 0)
    {
        result = result + "Recommendation: Add energy consumption records to begin analysis.";
    }
    else if(highUsageCount > 0)
    {
        result = result + "Recommendation: Review records above 150% of the average energy
consumption. Investigate lighting, air conditioning, and equipment usage.";
    }
    else
    {
        result = result + "Recommendation: No records exceed the high-consumption threshold. Continue
monitoring energy usage.";
    }
    return result;
}
```

C.12: getTotalEnergy (DS-11)

```
float getTotalEnergy()
{
    total_energy = Energy_Consumption.sum(Units_Consumed_kWh);
    return total_energy;
}
```

C.13: testOpenAIConnection (DS-12)

```
string testOpenAIConnection()
{
    response = invokeurl
    [
    url :"https://api.openai.com/v1/models"
    type :GET
    connection:"openai_sustainability_connection"
    ];
    return response.toString();
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Accept the invitation e-mail from Zoho Creator and sign in with the same e-mail address.
2. On the web, open creator.zoho.in, select the account kanishka.s.2024.cse_rajal703 if the account switcher shows another account, and open Smart Campus Sustainability & Carbon Tracker.
3. On a phone, install the Zoho Creator app, sign in with the same account and open the application.

D.2 Instructions for Administrators and Data Entry Staff

- Add buildings first: Buildings > Building. Enter the ID, name, block, department, floors, capacity and status.
- Record electricity: Energy Consumptions > Energy Consumption. Pick the building, date, meter reading, units (kWh) and cost. The estimated CO_2 and status are filled automatically.
- Record water: Water Consumptions > Water Consumption. Enter litres used, source and cost.
- Record waste: Waste Management > Waste Management. Enter total and recycled quantity; the recycling percentage is calculated for you.
- Log activities: Green Activities > Green Activities, then record each student in Student Participations.
- Review: open the reports in each section and the Sustainability Dashboard for the summary.

D.3 Instructions for Students and Staff

- Report a problem: Sustainability Complaints > Sustainability Complaints. Choose the category and location and describe the issue.
- Ask a question: Campus Sustainability AI Assistant. Type a question such as “How can we save water?” and open the AI Assistant report to read the answer.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	KANISHKA S	Register No.	240701227
Project Title	Smart Campus Sustainability & Carbon Tracker	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	KANISHKA S	Register No.	240701227
Department / Section	CSE	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____